Applied Soft Computing Journal 149 (2023) 111004

Contents lists available at ScienceDirect

Applied Soft Computing

journal homepage: www.elsevier.com/locate/asoc

Volleyball premier league algorithm with ACO and ALNS for simultaneous pickup–delivery location routing problem

Shuo Sun *, Liang Ma, Yong Liu, Chunjian Shang

School of Management, University of Shanghai for Science and Technology, China

ARTICLE INFO

Keywords:
Reverse logistics
Location routing problem
Simultaneous pickup and delivery
Hybrid VPL algorithm

ABSTRACT

The reverse logistics problem, which involves customer returns and incurs high additional logistics costs, has become a topic of increasing interest. This problem is modeled as the location routing problem with simultaneous pickup and delivery under time constraints (LRPSPDTW), which is a more complex version of the location routing problem (LRP). Previous studies have shown that existing algorithms are unable to provide satisfactory solution accuracy for this problem. In order to address this, a novel algorithm called the hybrid volleyball premier league (HVPL) algorithm is proposed. Drawing inspiration from the competition mechanism of the original volleyball premier league (VPL) algorithm, the HVPL algorithm incorporates adaptive large neighborhood search (ALNS) and improved ant colony optimization (ACO) as sub-algorithms, allowing them to work together synergistically and strike a balance between exploration and exploitation. The learning and elimination phases of the VPL algorithm have been redesigned to enhance convergence speed. Furthermore, the Bellman's algorithm has been enhanced to determine depot locations and construct vehicle paths for the solution. Prior to solving the LRPSPDTW problem, benchmark testing was conducted using 35 latest benchmark functions. Experimental results and Friedman mean rank test demonstrated that HVPL exhibits superior exploration and exploitation capabilities. HVPL was subsequently tested on 66 instances based on the modified Solomon's benchmark and the NEO research group benchmarks, and the results showcased its outperformance over other methods in the majority of cases.

1. Introduction

The location routing problem (LRP) is a classical combinatorial optimization problem, which is a combination of two optimization problems, namely locating depots and determining vehicle routes from these depots [1]. The idea of combining these two types of optimization problems was proposed almost sixty years ago, but was limited by the ability of computers to solve mathematical models of complex problems [2] until the late 1980s, when Salhi and Rand [3] achieved the first quantitative study of solving the facility siting problem while considering the route optimization problem. Their research shows that solving the vehicle routing problems and loacating depots problems separately usually leads to suboptimal solutions. The optimization objective of LRPs consists of three main components. First, to minimize the total cost of opening the facility by selecting the facilities to be opened from the candidate facilities; second, to determine from which open facility to serve which customer; and third, to minimize the travel cost incurred by the vehicle to serve the customer [4]. In fact, these three sub-objectives affect and constrain each other, so the problem should be considered and solved from the perspective of logistics network system. The LRP has different variants,such as Prize-Collecting LRPs (PCLRPs), Split delivery LRPs (SDLRPs), Pickup-and-delivery LRPs,Iventory LRPs (ILRPs), Multi-echelon LRPs [5].

Due to the need of modern complex logistics systems [6,7], another variant of LRP, the location routing problem with simultaneous pickup–delivery and time windows (LRPSPDTW), was proposed, which is the focus of this paper. In most of the previous studies on this problem, metaheuristic algorithms have been used to solve the LRPSPDTW, mainly because it is an NP-hard problem [8] and using metaheuristic algorithms can optimize the problem to provide satisfactory results within reasonable time constraints [9,10], and these algorithms have some general strategies that are not limited to solving specific problems. However, if exact methods are used, it requires an excessive amount of time to determine the optimal solution. The metaheuristic algorithms also have some limitations, such as premature convergence to a local optimum or even stagnation. To improve the computational performance of metaheuristic algorithms and to overcome these limitations, many studies have developed hybrid methods, which usually combine the advantages of different search approaches [11].

* Corresponding author.
E-mail address: 211420077@st.usst.edu.cn (S. Sun).

https://doi.org/10.1016/j.asoc.2023.111004
Received 14 March 2023; Received in revised form 6 October 2023; Accepted 29 October 2023
Available online 31 October 2023
1568-4946/© 2023 Published by Elsevier B.V.

Several studies have been conducted to apply the LRPSPDTW problem in certain scenarios and solve it using meta-heuristics algorithms. Wang et al. [12] studied the multi-objective location routing problem with simultaneous pickup and delivery in the context of urban distribution and used a hybrid heuristic algorithm combining tabu search and efficient procedures to solve. Qiu et al. [13] developed a multi-objective model for solving the problem of green location-routing with cold chain logistics and proposed an optimization framework using three multi-objective evolutionary algorithms and two search mechanisms. Leng et al. [14] addressed the regional low-carbon problem and proposed a bi-objective model with simultaneous pickup and delivery, hard time windows and a heterogeneous fleet. To solve the model, a novel hyper-heuristic method was developed that is based on a quantum approach as a high-level selection strategy and incorporates the great deluge, late acceptance and environmental selection as acceptance criteria. Wang et al. [15] also conducted a study for low-carbon logistics and designed a two-stage hybrid heuristic algorithm to solve the problem. In the paper, the initial path is firstly constructed by clustering customer points using a genetic algorithm, and then a local search is performed using a variable neighborhood algorithm, and the idea of simulated annealing algorithm is incorporated into the framework of the allometric neighborhood algorithm. Meanwhile, the vehicle adjustment strategy is added in the optimization process. Zhou et al. [16] discussed the e-commerce supply chain last mile delivery problem and proposed a model for the location routing problem with simultaneous home delivery and customer pickup, with the goal of constructing an efficient last mile distribution system. A hybrid evolutionary algorithm combining genetic algorithm and local search algorithm was developed for this problem, and a diverse population generation algorithm as well as a two-stage solution initialization heuristic is used to generate the initial population. Mamaghani et al. [17] studied the closed-loop supply chain for the retail industry. In the paper, this problem was modeled as a bi-objective periodic location routing problem with simultaneous pickup and delivery and soft time windows, and both NSGA-II and NRGA algorithms were used to solve the problem. However, since the problem belongs to the NP-hard problem, the existing algorithms often suffer from low solution accuracy or high memory cost. Therefore, designing more efficient methods for this problem remains an open and meaningful research topic.

To improve the solution accuracy, a novel metaheuristic discrete hybrid volleyball premier league algorithm (HVPL) for the LRPSPDTW problem based on original volleyball premier league (VPL) algorithm is proposed. In recent studies, many swarm-based algorithms inspired by sports have been proposed, which have attracted much attention because of their ability to converge more reliably and quickly to the global optimal solution. An optimization method called Soccer League Competition (SLC) algorithm for optimal design of water distribution networks was proposed by Naser Moosavian et al. [18]. The method was inspired by the competition and cooperation between teams and players in soccer leagues. It simulates soccer games to find the global optimal solution by continuously updating the solution vectors. The SLC algorithm was compared with other meta-heuristic methods on three benchmark pipe networks and showed that SLC is more reliable, faster, and more efficient. Worawattawechai et al. [19] developed a learning enhanced golden ball algorithm (LEGBA) for solving vehicle routing problems. It uses the concept of soccer leagues where each player represents a solution. The algorithm performed well on 45 standard test problems, achieving or exceeding the best-known level in 40 problems and creating new best records in 20 problems. Bouchekara [20] proposed a new optimization algorithm called the Most Valuable Player Algorithm (MVPA), which is inspired by the competition and cooperation between players in sports. The author evaluated the performance of MVPA using 100 test functions and compared it with 13 well-known optimization algorithms. The results showed that MVPA has the best success rate and the lowest function evaluation times in all experiments. The League Championship Algorithm (LCA) is a new evolutionary algorithm presented by Kashan [21] that mimics the competitive environment of sports leagues. It uses artificial teams as potential solutions and updates them based on the outcomes of matches and seasons. It was evaluated on a large number of benchmark functions and compared with three other algorithms. It showed better quality of final solutions and convergence speed than the other algorithms. A new swarm intelligence algorithm based on group behavior and some artificial physical processes, which mimics the interaction behavior among teams in a volleyball league, was proposed by Moghdani et al. [22]. A modified version of this algorithm was proposed in [23], where the sine cosine algorithm (SCA) is introduced in the learning phase to avoid the original VPL from falling into a optimum local solution. In the study by Prakash et al. [24], the algorithm was used to obtain different optimization parameters for the proposed controller, allowing the controller to obtain a more stable dynamic performance in different power transaction modes of the restructured environment. In [25], a multi-objective version of this algorithm was extended for obtaining the best solution for a given problem with multiple contradicted objectives, and this multi-objective version has high implementability in engineering design optimization problems. Salimifard et al. [26] presented an alternative multi-objective version of this algorithm that introduces the concept of using crowding distance in NSGA-II and used five standard metrics and ten test problems to show the performance advantages of this version. Elaziz et al. [27] introduced the whale optimization algorithm (WOA) into the learning phase of this algorithm to be used as a local search system and applied this improved version to determine the optimal threshold for image segmentation, demonstrating superiority in several performance metrics. Murali et al. [28,29] proposed to introduce position-based learning (OBL) into this algorithm and successfully applied it to adjust the parameters of the optimal intelligence-based controller.

From the research related to the VPL, it is known that many strategies can be embedded in this framework to improve the global optimization capability based on the concept of competition and interaction. Thus, the algorithm has a significant advantage in solving engineering problems. Table 1 presents a comprehensive overview of the optimization problems that have been addressed by the VPL algorithm and its extensions. Additionally, it highlights the comparison algorithms that have been outperformed by either the VPL algorithm or its extensions. This analysis serves to demonstrate the superior performance and effectiveness of the VPL algorithm and its extensions in solving these optimization problems. It is important to note that previous studies have primarily focused on utilizing VPL for continuous optimization problems, such as parameter tuning for controllers. The search mechanism of the VPL involves multiple real vector operations to update specific element values, which means that the original VPL is not suitable for solving discrete optimization problems like LRPSDPTW.

In this paper, we propose a novel approach to enhance the original VPL by redesigning suitable strategies. Building upon the competitive framework of the HVPL, we integrate the widely-utilized ACO and ALNS algorithms for hybridization into this framework. By incorporating the ACO algorithm with its global search mechanism and the ALNS algorithm with its neighborhood search mechanism, we significantly broaden the search solution space and effectively maintain a balance between exploration and exploitation of the algorithm. This paper contributes in the following respects:

1. The original VPL algorithm has been extended by redesigning strategies and mechanisms to solve combinatorial optimization problems such as LRPSPDTW.

2. The improved Bellman's algorithm has been used to determine the depot locations and constructing vehicle routes. Additionally, HVPL incorporates ALNS and ACO with different search mechanisms as sub-algorithms embedded in the HVPL framework, enhancing the algorithm's ability to search the solution space and improving overall performance. To prevent premature convergence of the ant colony

Table 1
Applications of VPL algorithm and its variants.

VPL and its variants	Optimization problems	Comparison Algorithm	Ref.
VPL	Optimize the parameters of 2-DOF-PI-FOPDN controller.	PSO, FA, DEA	[24]
VPL	Optimize the parameters of the LFC mechanism.	PSO, WOA	[28]
VPL	Scheduling parallel machines.	MOPSO, NSGA-II, MOGWO, MOALO, MOEA/D, SPEA2	[26]
VPLWOA	Multilevel thresholding image segmentation.	FA, SCA, SSO, VPL, WOA	[27]
OVPL	Optimize the parameters of CC-2DOF(PI)-PDF controller.	PSO, TLBO, VPL	[29]

algorithm, two local search operators have been designed, namely repositioning strategy and substitution strategy.

3. The proposed algorithm has been tested and evaluated on 35 benchmark functions and 66 instances generated from Solomon's benchmarks and the NEO Research Group benchmarks. The 35 benchmark functions include unimodal, multimodal, and CEC-BC-2017 benchmark functions. A comparison has been made with seven advanced algorithms, including the original VPL algorithm. Experimental results and Friedman mean rank test demonstrate that HVPL exhibits excellent optimization capabilities in both unimodal and multimodal functions. HVPL was also tested on 66 LRPSPDTW instances against seven other algorithms commonly used to solve combination optimization problems. The outstanding performance of the proposed algorithm on 58 instances and the Wilcoxon Signed-Rank test both confirm HVPL's competitive advantage in solving real-world optimization problems.

The rest of the paper is organized as follows: Section 2 presents a mathematical model for the LRPSPDTW problem. In Section 3, we outline the implementation details of the algorithm proposed in this paper. To assess the performance of proposed algorithm, Section 4 presents computational experiments using 35 latest benchmark functions and 66 LRPSPDTW instances. Finally, in Section 5 provides a concise summary and discuss potential future research directions.

2. The location-routing problem with simultaneous pickup–delivery and time windows

Let $G = (V, E, \lambda)$ be a weighted and undirected graph, where $V = C \bigcup D$, C represents the nodes of customers, D represents the nodes of potential depots, E is a set of arcs $E = \{(i, j) : i, j \in V\}$ and λ denotes the set of traveling costs associated with the set of arcs E. Each customer has a pickup and delivery requirements, and each customer can only be serviced once. We consider the case of homogeneous vehicles, which means that the maximum capacity and fixed maintenance costs are the same for each vehicle, and the vehicles return to the same depots after completing their pickup and distribution tasks. In addition, there is a maximum capacity limit for all opened depots. The objective of the LRPSPDTW is to minimize the total cost of the system, including depot opening costs, fixed costs and variable costs of vehicles. The definitions and descriptions of the various sets, parameters, and decision variables adopted in the mathematical model are shown in Tables 2 to 4, and the mathematical model of the problem is given below.

$$min\, C_T = \sum_{i\in V}\sum_{j\in V} c_{ij}x_{ij} + \sum_{j\in D}\phi_j y_j + \sum_{i\in C}\sum_{j\in D}\eta x_{ij} \tag{1}$$

$$\sum_{j\in V} x_{ij} = 1 \quad \forall i \in C \tag{2}$$

$$\sum_{i\in V} x_{ij} - \sum_{i\in V} x_{ji} = 0 \quad \forall j \in V \tag{3}$$

$$\sum_{j\in V} \alpha_{ji} - \sum_{j\in V} \alpha_{ij} = d_i \quad \forall i \in C \tag{4}$$

$$\sum_{j\in V} \beta_{ij} - \sum_{j\in V} \beta_{ji} = p_i \quad \forall i \in C \tag{5}$$

$$\alpha_{ij} + \beta_{ij} \le \rho x_{ij} \quad \forall i, j \in V, i \ne j \tag{6}$$

Table 2
Sets.

Notation	Definition
C	Set of customers
D	Set of depots
V = C ⋃ D	Set of all nodes

$$\sum_{j\in C} \alpha_{kj} = \sum_{j\in C} z_{jk}d_j \quad \forall k \in D \tag{7}$$

$$\sum_{j\in C} \alpha_{jk} = 0 \quad \forall k \in D \tag{8}$$

$$\sum_{j\in C} \beta_{jk} = \sum_{j\in C} z_{jk}p_j \quad \forall k \in D \tag{9}$$

$$\sum_{j\in C} \beta_{kj} = 0 \quad \forall k \in D \tag{10}$$

$$\alpha_{ij} \le (\rho - d_i)x_{ij} \quad \forall i \in C, \forall j \in V \tag{11}$$

$$\beta_{ij} \le (\rho - p_i)x_{ij} \quad \forall i \in V, \forall j \in C \tag{12}$$

$$\alpha_{ij} \ge d_j x_{ij} \quad \forall i \in V, \forall j \in C \tag{13}$$

$$\beta_{ij} \ge p_i x_{ij} \quad \forall i \in C, \forall j \in V \tag{14}$$

$$\sum_{k\in D} z_{ik} = 1 \quad \forall i \in C \tag{15}$$

$$\sum_{i\in C} d_i z_{ik} \le \psi_k y_k \quad \forall k \in D \tag{16}$$

$$\sum_{i\in C} p_i z_{ik} \le \psi_k y_k \quad \forall k \in D \tag{17}$$

$$x_{ik} \le z_{ik} \quad \forall i \in C, \forall k \in D \tag{18}$$

$$x_{ki} \le z_{ik} \quad \forall i \in C, \forall k \in D \tag{19}$$

$$x_{ij} + z_{ik} + \sum_{\substack{m\in D\\ m\ne k}} z_{jm} \le 2 \quad \forall i, j \in C, i \ne j, \forall k \in D \tag{20}$$

$$e_i(\sum_{j\in V} x_{ij}) \le \mu_i \le l_i(\sum_{j\in V} x_{ji}) \quad \forall i \in V \tag{21}$$

$$\mu_i + t_i + \tau_{ij} - \mu_j \le M(1 - x_{ij}) \quad \forall i, j \in V \tag{22}$$

The objective function (1) represents the total cost of this logistics system, including transportation costs, opening depot costs, and vehicle fixed costs. Constraint (2) guarantees that each customer is serviced once and only once. Constraint (3) ensures that vehicles are balanced in and out of the path. Constraint (4)–(5) eliminate subtour and ensure that every customer's pickup and delivery needs are satisfied. Constraint (6) ensures that the total load does not exceed the vehicle capacity. Constraint (7) ensures that the total delivery load of the depot is equal to the sum of the delivery demands of all customers served by the depot. Constraint (8) ensures that all vehicles departing from the depot deliver all goods to customers and return them to the depot.

Table 3
Parameters.

Notation	Definition
c_{ij}	Cost of travel between nodes i and j, $i,j \in V$
d_i	Delivery quantity for customer i , $i \in C$
p_i	Pickup quantity for customer i , $i \in C$
t_i	Service time for customer i , $i \in C$
e_i	Lower bound of the time window for customer i , $i \in C$
l_i	Upper bound of the time window for customer i , $i \in C$
τ_{ij}	Time of travel between nodes i and j , $i,j \in V$
ϕ_j	Fixed cost of opening depot j, $j \in D$
η	Fixed cost of using a vehicle
ψ_j	Capacity of a depot j , $j \in D$
ρ	Capacity of a vehicle
M	A large number

Table 4
Decision variables.

Notation	Definition
x_{ij}	If a vehicle travels from node i to node j,then $x_{ij} = 1$, and $x_{ij} = 0$ otherwise , $i,j \in V$
y_j	If depot j is open,then $y_j = 1$, and $y_j = 0$ otherwise , $j \in D$
z_{ij}	If customer i is assigned to depot j,then $z_{ij} = 1$, and $z_{ij} = 0$ otherwise , $i \in C$, $j \in D$
μ_i	Arrive time of a vehicle at customer i, $i \in C$
α_{ij}	Remaining delivery demands after leaving node i if a vehicle travels from node i to node j, then $\alpha_{ij} > 0$,and $\alpha_{ij} = 0$ otherwise , $i,j \in V$
β_{ij}	Cumulated pickup up demands up to node i if a vehicle travels from node i to node j, then $\beta_{ij} > 0$,and $\beta_{ij} = 0$ otherwise , $i,j \in V$

Constraint (9) ensures that all pickup load received by the depot is equal to the sum of all customers' pickup demand. Constraint (10) ensures that all pickup demands of the depot are from the customers who need to be served by this depot. Constraints (11)–(14) are the boundary constraints for additional variables. Constraint (15) ensures that each customer is served once by one depot only. Constraint (16)–(17) ensure that total delivery or pickup demands of customers do not exceed the maximum capacity of the depot. Constraint (18)–(19) prohibit the assignment of customers to unopened depots. Constraint (20) ensures that two consecutive customers on a route are assigned to the same depot. Constraint (18)–(20) removal of routes that do not start from and return to a depot. Constraint (21) guarantees time windows for customers and depots requirements. Constraint (22) ensures continuity of vehicle travel time.

3. Proposed method

3.1. Volleyball premier league algorithm

The volleyball league algorithm [22] is inspired by the activities between teams in volleyball matches, and its core concept is to implement novel strategies by mimicking the interactions between teams in a volleyball league and capturing these interactions. In the VPL algorithm, each individual (solution) in the population is represented by a team, and each solution consists of two parts, the formation and the substitution. The objective function of the solution is determined by the formation.

Since the algorithm was designed based on the concept of the volleyball league, a league schedule needed to be developed. A method known as single round robin (SRR) was chosen as the competition schedule. According to this league schedule, and assuming that there are N teams in the league, each team plays only once a week, each team plays $(N-1)$ games during the tournament. Assuming that there are eight teams in the league, denoted by capital letters A through H, and each team plays one game per week on a regular basis, the schedule

Table 5
A league schedule.

Week 1	Week 2	Week 3	Week 4	Week 5	Week 6	Week 7
A–H	A–G	A–F	A–E	A–D	A–H	A–B
B–G	B–E	B–C	B–H	B–F	B–G	C–H
C–F	C–D	D–H	C–G	C–E	C–F	D–C
D–E	F–H	E–G	D–F	G–H	D–E	E–F

of each team against all other teams during the tournament is shown in Table 5.

In order to determine the strength of each team during the week and the formation of the team i defined as X_i^f, a strength index is introduced as follows:

$$\varphi(i) = \frac{f(X_i)}{Z}, \tag{23}$$

$$Z = \sum_{i=1}^{n} f(X_i), \tag{24}$$

where $\varphi(i)$ is the strength index value of team i, $f(X_i) = 1/C_T$ indicates the objective function value of team i and Z denotes the sum of the objective function values of all teams in the league for a week. Let team j play against team k. The probability $P(j,k)$ of team j winning is calculated as follows:

$$P(j,k) = \frac{\varphi(j)}{\varphi(j) + \varphi(k)}. \tag{25}$$

There is no tie in a volleyball match, only loss or win for each team. A uniformly distributed random number $r \in [0,1]$ is used to determine the winner and loser, if $r < P(j,k)$, then team j wins, otherwise team k is the winner. After determining the winner and loser of the match, the two teams will use the winning or losing strategies to improve their strengths respectively. From the algorithmic point of view is to find the optimal solution by searching the solution space using these strategies.

To accelerate the convergence of the algorithm, the learning phase and elimination phase are designed. During the learning phase, the teams are sorted in order of smallest to largest objective function values, and then the top three teams are named rank 1, rank 2 and rank 3, which are seen as models for other teams to follow, and the other teams randomly choose one of the models to learn to improve their own strength. The details are described in Section 3.2.6. In the elimination phase, the lower ranked $N\delta$ teams are eliminated according to the proportional δ and an equal number of teams are added to the league according to certain rules. After this phase, the quality of the solutions in the population is further enhanced. The details will be depicted in Section 3.2.7.

3.2. Discrete hybrid VPL

A method for solving the combinatorial optimization problem, called hybrid volleyball premier league (HVPL), was proposed that redesigned the individual encoding mechanism and update strategies of the original volleyball premier league (VPL). There are two reasons why we tried to develop this algorithm. First, the accuracy of the existing algorithms for solving this problem cannot be further improved, so a novel metaheuristic algorithm is used. And the original VPL algorithm has good convergence and stability, its structure is easy to integrate with other optimization algorithms [28,29]. Second, hybridization allows two or more algorithms to work in concert, expanding the search solution space [30]. The structure of the HVPL algorithm is illustrated in Fig. 2. During algorithm execution, the teams play according to the league schedule SRR (introduced in Section 3.1) and the Eq. (25) is used to determine the winner and loser of each match. The losing team takes the ACO for post-game training, and the winning team uses the ALNS for training. The strengths of the two teams are evaluated after training, if the losing team does not outperform the winning team,

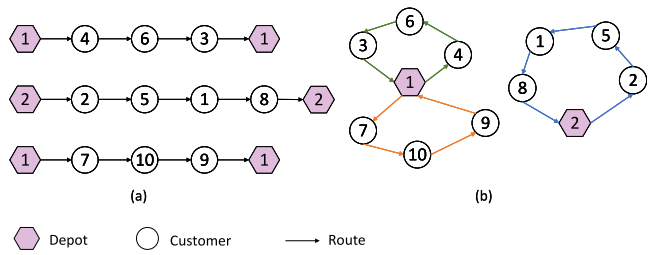


Fig. 1. Example of a solution, (a) the routes representation (b) the graphical representation of a solution.

the losing team uses a repositioning strategy (shown in Section 3.2.3) to improve its strength, and vice versa using a substitution strategy (shown in Section 3.2.4) to train again. The pseudo-code to determine the winner of the match and how the winning and losing teams adopt the training strategies after the match is shown in Algorithm 1. From the algorithmic point of view, the repositioning strategy or substitution strategy is used as local search operator to avoid premature convergence and to improve the quality of solutions generated by the sub-algorithm ACO. This process is repeated until all the seasons are finished.

Different from other hybrid metaheuristics, the HVPL is a parallel hybrid algorithm that ensured a balance between exploration and exploitation. It embeds the ALNS with strong adaptability in neighborhood search [31] and the ACO with global search [32,33] as sub-algorithms into the framework of the HVPL. The two sub-algorithms simultaneously implemented different types of search strategies for some individuals in the population according to the SRR. This combination allows the ALNS and the ACO to play a complementary and synergistic role, thus providing a rich diversity in the exploration of the system and increasing its exploitation capability. In addition, two local search strategies are developed to improve the quality of the ACO solutions. The improved Bellman's algorithm [34] is used to construct solutions throughout the whole execution process, and this method avoids generating a large number of infeasible solutions. In the learning phase, the top three best solutions of the population are taken as learning objects, and the other solutions rebuild their structures based on them, so that the overall solution quality became better and better, achieving a positive cycle. In the elimination phase, individuals with poor solution quality are removed and new individuals with the similar strength level to the ones that remained are added. This strategy not only accelerates the convergence of the algorithm, but also further improves the overall quality of the solutions. The following subsections describe the main steps of the HVPL algorithm in detail.

Algorithm 1 Competition (i , j)

Calculate $\varphi(i)$ and Z using Eqs (23) and (24).
Calculate $P(j,k)$ using Eq (25).
Generate $r \in [0,1]$.
if $P(j,k) \le r$ **then**
    team i is the winner and team j is the loser ;
else
    team j is the winner and team i is the loser ;
end if
Apply the ALNS for the winner.
Apply the ACO for the loser.
if $f(X_{winner}) < f(X_{loser})$ **then**
    apply substitution strategy for the loser ;
else
    apply repositioning strategy for the loser ;
end if
Update the best team.

Table 6
Parameters of Algorithm 2.

Notation	Definition
n	Number of customers
V_i	The cost of the shortest path from node 0 to node i
$load_i$	Vehicle load at node i
$cost$	The trip cost
lv_i	Time of vehicle leaving node i
P_i	The predecessor of node i on this path

3.2.1. Solution representation and initial population

In the HVPL, a solution was represented by multiple sub-routes. Fig. 1 provides an example with 10 customers and 2 depots. The vehicle routes are represented in Fig. 1(a), where there are customers 3, 4, 6, 7, 9, 10 are assigned to depot 1, and the depot needs to schedule two vehicles to serve the customers, and customers 1, 2, 5, 8 are served by depot 2, and only one vehicle is needed to satisfy the pickup and delivery services. The graphical representation of the solution is provided in Fig. 1(b).

The Bellman's algorithm proposed by Beasley [35], which is often used to find the shortest path, was used to construct each solution in the initial population step by step after adding the constraints of the problem studied in this paper. The algorithm consists of the following steps: (1) Randomly generate a sequence of n customer node numbers S_c, as shown in Fig. 3 (Step 1). (2) The customer nodes on each shortest path (sequence of customer nodes for sub-route) are obtained by splitting the S_c using Bellman's algorithm generating two labels V_j and P_j for each customer node. Algorithm 2 illustrates the specific steps of the algorithm for splitting the paths, the notations in the algorithm are shown in Table 6, and others have been represented in Tables 3 and 4. (3) Each sub-route is assigned to the nearest depot by calculating the distance from the starting customer point to each candidate depot. The initial solutions generation process is shown in Fig. 3.

Algorithm 2 Split routes

$V_0 := 0$ // The subscript 0 means depot
for i:= 1 to n **do**
    $V_i := +\infty$
end for
for i:= 1 to n **do**
    $load_0 := \rho; cost := 0; j := i; lv_0 := 0$
    while stopping conditions are not satisfied **do**
        $load_j := load_{j-1} - (d_j + q_j)x_{ij}$
        if $i = j$ **then**
            $\mu_j = max(e_j, e_0 + \tau_{0j})$
            $lv_j = \mu_j + t_j$
            $cost := cost - c_{0,j}x_{0,j} + c_{j,0}x_{j,0}$
        else
            $\mu_j = max(e_j, lv_{j-1} + \tau_{j-1,j})$
            $lv_j = \mu_j + t_j$
            $cost := cost - c_{j-1,0}x_{j-1,0} + c_{j-1,j}x_{j-1,j} + c_{j,0}x_{j,0}$
        end if
        if $(0 \le load_j \le \rho)$ and $(lv_j \le l_j)$ and $(lv_j + \tau_{j,0} \le l_0)$ **then**
            if $V_{i-1} + cost < V_j$ **then**
                $V_j := V_{i-1} + cost$
                $P_j := i - 1$
            end if
            $j := j + 1$
        end if
        until $(j > n)$ or $(load_j > \rho)$ or $(lv_j > l_j)$ or $(lv_j + \tau_{j,0} > l_0)$
    end while
end for

3.2.2. The ACO sub-algorithm

The ant colony optimization (ACO) was first proposed by Dorigo et al. [36] to solve the classical traveling salesman problem (TSP). In

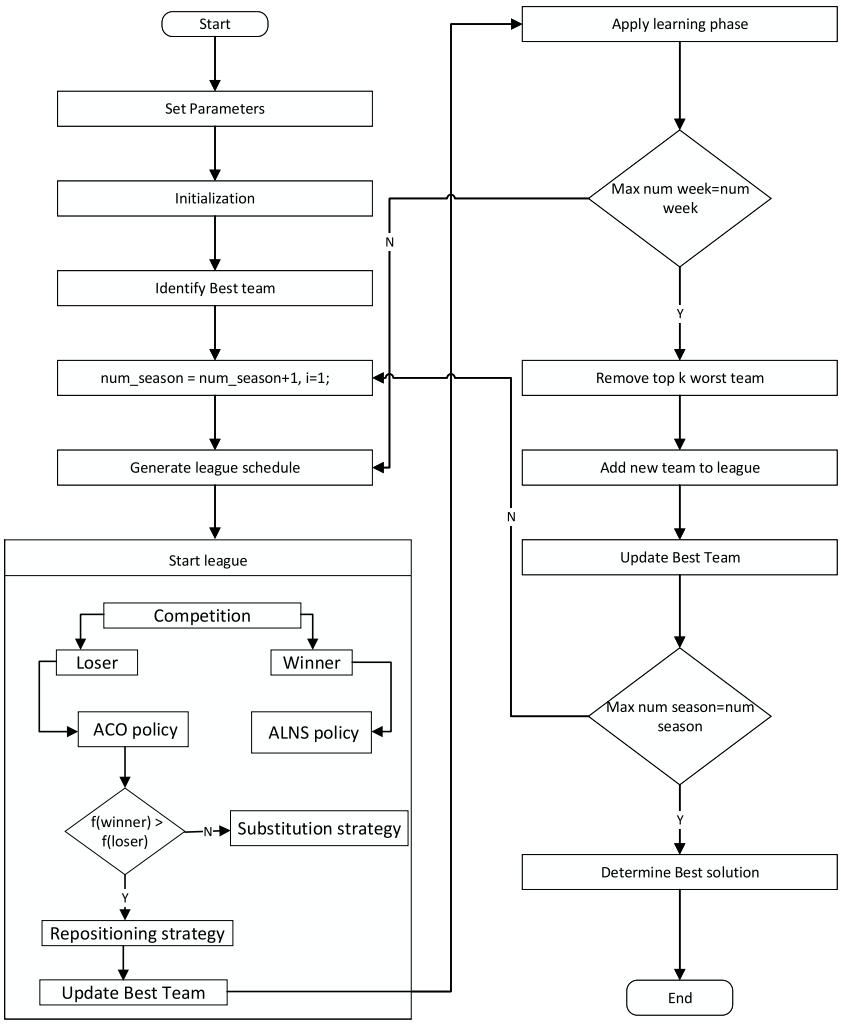


Fig. 2. Flowchart of the HVPL algorithm.

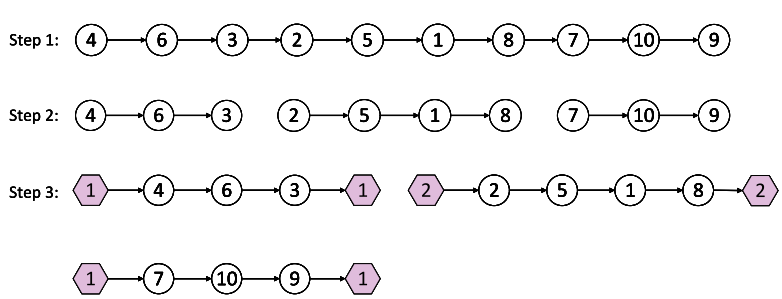


Fig. 3. An initial solution construction steps: (Step 1) Generate a random sequence of customer numbers, (Step 2) Split the sequence using Bellman's algorithm, (Step 3) Assign the nearest depot to each subpath.

this paper, this method was chosen as the sub-algorithm of the HVPL mainly because it has distributed characteristics, each ant is independent and has a strong global search capability. The basic principle of ACO for solving the vehicle routing problem is that it released a substance called "pheromone" on the path it traveled. As time progresses, the shorter paths deposit more pheromone and have a higher probability of being selected for subsequent choices, so the number of ants choosing that path increases, resulting in higher concentrations of pheromone accumulating on the shorter paths. Eventually, the path with the highest concentration is the best path [37,38]. In this paper, the failed team updates the customer sequence by the ACO method, and subsequently uses Bellman's algorithm to split the customer sequence according to the constraints, and then each subroute is assigned the depot closest to it. The main process is as follows :

(a) Each ant randomly selects one customer at a time from the S_c to progressively construct a sequence of customers according to the biased probability rule, which is as follows:

$$P_{ij} = \begin{cases} \frac{\tau_{ij}^{\alpha} \cdot \eta_{ij}^{\beta}}{\sum_{l \in allowed} \tau_{il}^{\alpha} \cdot \eta_{il}^{\beta}} & , l \in allowed \\ 0 & , \text{otherwise} \end{cases} \tag{26}$$

where, i, j denote the starting and ending points of edges, respectively. $allowed$ represents the set of unvisited customer nodes. τ_{ij} denotes the pheromone concentration of edge(i, j). η_{ij} is the heuristic information of the edge(i, j), which is defined as $\eta_{ij} = \frac{1}{d_{ij}}$, d_{ij} is the length of the edge(i, j). α and β control the relative importance of τ_{ij} and η_{ij}, respectively.

(b) The customer sequence is split using the Algorithm 2 to obtain all partial solutions s^p of a complete solution.

(c) Calculate the objective function value of the solution $f(X_i^f)$.

(d) When the season ends, the pheromone trails are updated as follows:

$$\tau_{ij} = (1 - \rho) \cdot \tau_{ij} + \sum_{k=1}^{m} \Delta\tau_{ij}^{k} \tag{27}$$

$$\Delta\tau_{ij}^{k} = \begin{cases} \frac{1}{L_k} & , \text{the ant k pass through the edge } (i, j) \\ 0 & , \text{otherwise} \end{cases} \tag{28}$$

where $\rho \in (0, 1)$ is the volatility factor of the pheromone. $\Delta\tau_{ij}^{k}$ is the amount of pheromone released on the edge (i, j) that ant k passed through. L_k is the length of the path constructed by ant k

Algorithm 3 shows the pseudo-code of the ACO search operator.

Algorithm 3 The ACO sub-algorithm

```
Set parameters, initialize pheromones trails.
Ant based construct a sequence of customers using Eq (26).
Obtain all partial solutions using Algorithm 2.
Calculate f(X_loser) using Eq (1).
if max_num_week = num_week then
    pheromone update using Eqs (27) and (28).
end if
```

3.2.3. Repositioning strategy

Local optimization strategies have been widely shown to improve the search performance of the ACO well. Dewantoro et al. [39] combined Tabu Search Algorithm as a local search with the ACO to solve the traveling salesman problem. Wu et al. [40] proposed a hybrid algorithm that combines the ACO and a hash tabu method as a local search operator to solve the winner determination problem. Al-behadili et al. [41] presented a hybrid iterated local search and the ACO to improve classification accuracy and the size of the classification model. In order to improve the quality of solutions generated by failed teams using the ACO, this subsection introduces a local search strategy called repositioning strategy.

The design of the local search strategy was inspired by the losing team behavior during post-game practice. The team's coach rearranged the players based on their performance in the previous game to put them in more suitable positions for better performance. The steps are as follows:

(a) According to the total number of customers n, randomly select $N_{rp} \in (0, n)$ positions from the S_c to form a list l_{rp}.

(b) Iterate through each number i in l_{rp}, and randomly select a customer number j from S_c. When the number i and j are different, the two customer numbers exchange positions.

(c) After all the numbers in l_{rp} have been iterated, a solution is obtained using the Algorithm 2 and its objective function value $f(X_i^f)$ is calculated.

Algorithm 4 illustrates the implementation of the repositioning strategy.

Algorithm 4 Repositioning strategy

```
Select randomly N_rp positions to form l_rp.
for i in l_rp do
    select randomly a position j from S_c,
    if i ≠ j then
        reverse two positions i and j.
    end if
end for
Using Algorithm 2 to get all partial solutions.
Calculate f(X_loser) using Eq (1).
```

3.2.4. Substitution strategy

Since the local optimum is differs across various neighborhood structures, using multiple neighborhood search operators can improve the chances of finding a more satisfactory local optimum [4]. To expand the space of prospective solutions and effect significant changes to current solutions, a substitution strategy different from the original VPL algorithm is proposed. To quickly boost the losing team's strength (solution), the coach of the team substitutes the current best team's formation instead of merely swapping positions within the team between the formation and the substitution. The coach this by directly hiring some players from the current best team to replace the losing team's original players, thereby improving the team's strength rapidly. The coach has adopted the ACO before, but since it did not bring about significant improvement in the team's strength, he opted for this strategy in order to get a better result in the next match.

As described above, in order to make larger changes in the current solution, multiple customer nodes of the losing team's S_c are substituted by this local search operator with the customer nodes of the best team S_c^{best} in the same position. The specific steps are as follows:

(a) Select two numbers $I_1, I_2 \in [1, n]$ randomly, and $I_1 < I_2$.

(b) Intercept the segment l_m of the S_c from I_1 to I_2.

(c) Generate two empty lists l_f and l_b.

(d) Iterate through each position i in S_c. If the customer in position i of S_c^{best} is not in l_m, add the customer in position i to l_f or l_b.

(e) After traversing all positions in S_c, get l_f and l_b after being updated.

(f) Combine l_f, l_m and l_b into a new customer sequence S_c' and replace S_c.

(g) Obtain all partial solutions using Bellman's algorithm and calculate $f(X_{loser}^f)$ using Eq. (1).

The pseudocode for the substitution strategy is described in Algorithm 5.

Algorithm 5 Substitution strategy

```
Select two numbers I_1, I_2 ∈ [1, n] randomly, and I_1 < I_2.
l_m ← S_c[I_1 : I_2]    // Get the middle part of the S_c
l_f, l_b ← empty lists
for i in n do
    if S_c^best[i] not in l_m then
        if len(l_f) < I_1 then    // If the length of l_f is less than I_1
            add S_c^best[i] to l_f ;
        else
            add S_c^best[i] to l_b.
        end if
    end if
end for
S_c' ← [l_f : l_m : l_b]    // Combine the three lists to generate S_c'
S_c ← S_c'    // S_c' substitutes S_c
Using Algorithm 2 to get all partial solutions.
Calculate f(X_loser) using Eq (1).
```

3.2.5. The ALNS sub-algorithm

The ALNS has the advantages of easy extension, strong exploration ability, and strong adaptability of neighborhood search, which can better avoid falling into local extremes in the process of finding the optimum [42]. In contrast, the ACO has a strong global search ability, but it is easy to fall into local optimum. Therefore, by embedding the ALNS as the sub-algorithm in the HVPL algorithm framework, the advantages of the two sub-algorithms can complement each other and the disadvantages can cancel each other out. Based on the framework of the HVPL, these sub-algorithms play a synergistic role and the overall search performance of the algorithm is improved.

The winning team used the policy in post-match training. First, the weights ω_p and scores π_p of the every operator p are initialized. Then, a pair of operators, a removal operator and an insertion operator, is selected using a roulette wheel [43] according to a probability formula $\omega_p / \sum_{i=1}^{h} \omega_i$, from which it can be seen that the higher the weight of the operator, the higher the probability of being selected. In each iteration, an intrusive customer sequence S_c^d is obtained by using a removal operator, followed by an insertion operator to obtain a new S_c^{new}. The acceptance criterion is used to decide whether to accept a new solution and replace the current one, which is based on the simulated annealing algorithm [44]. This criterion has two main parameters, one is the current temperature $T > 0$, and the other is the cooling rate ζ. As the algorithm iterates, the initial temperature decreases by the equation $T = \zeta T_0$, T_0 is the initial temperature. In this paper, the new solution is accepted in two cases, one is that the new solution's objective function value is less than that of the current solution, and the other is to calculate the acceptance probability $exp(-\frac{f(X_{winner}^{new})-f(X_{winner}^{curr})}{T})$ and generate a random number $rand \in [0,1]$. If $exp(-\frac{f(X_{winner}^{new})-f(X_{winner}^{curr})}{T}) > rand$, then the new solution is accepted. Finally, the weight and score of each candidate operator are updated, which are given in Eqs. (29) and (30). The details of this strategy are shown in Algorithm 6.

$$\omega_p = (1-\chi)\omega_p + \frac{\chi \pi_p}{\varepsilon_p} \tag{29}$$

$$\pi_p^{\gamma+1} = \pi_p^{\gamma} + \begin{cases} \sigma_1 & ;\text{if } f(X_{winner}^{new}) < f(X^{best}) \\ \sigma_2 & ;\text{if } f(X^{best}) < f(X_{winner}^{new}) < f(X_{winner}^{curr}) \\ \sigma_3 & ;\text{if } f(X_{winner}^{new}) > f(X_{winner}^{curr}) \end{cases} \tag{30}$$

where the constants $\chi \in [0,1]$. π_p denotes the score obtained by the operator p. ε_p indicates the number of times the operator p is selected. π_p represents the score obtained by the operator p under the corresponding condition. γ denotes the number of iterations of this strategy in each game. σ_1, σ_2, σ_3 are constants indicating the score obtained by the operator p in the corresponding condition. The core of the ALNS is removal operators and insertion operators. In this study, two removal operators and two insertion operators are used. The details of the operators are described as follows:

Removal operators

- Random removal
  This operator randomly selects m customers from the customer number S_c by a certain percentage λ and removes λS_c customers from the corresponding routes to increase the diversity.
- Worst removal
  The main purpose of the operator is to minimize the total vehicle travel distance. Select a customer node from S_c, calculate and save the reduction value of the path distance before and after the node is deleted, and repeat this process for the remaining customer nodes to obtain the reduction value of the path distance before and after each customer node is deleted, sort the reduction value of the distance of all customer nodes in descending order, and select the top λS_c customer nodes with larger reduction values from the path according to a certain proportion λ [45].

Insertion operators

- Greedy insertion
  This method randomly selects a customer point to be inserted, calculates the distance increment before and after the customer point is inserted into each insertable position, and selects the position with the smallest distance increment to insert the customer point. The above process is repeated until all the customer points to be inserted are added to the path [45].
- Regret insertion
  The operator uses the 2-regret criterion [45]. Calculate the distance increment Δd_i generated by each customer point after insertion at all insertable positions, and calculate the difference $\Delta d_i^2 - \Delta d_i^1$ between the two values with the smallest distance increment, whose superscripts 1 and 2 denote the optimal and suboptimal insertion positions of the customer, respectively. The customer point i^* with the largest difference value is found and inserted into the path.

Algorithm 6 The ALNS sub-algorithm

Input: $\omega_p = (1,...,1), \pi_p = (0,...,0), \gamma, T_0, \chi, \sigma_1, \sigma_2, \sigma_3, \zeta$,some removal operators R and insertion operators I
Output: global best solution, s_c^{best}

```
T ← T_0
for j := 1 to γ do
    r* ← SelectRemoval(R)
    i* ← SelectInsertion(I)
    S_c^d ← r*(S_c)
    S_c^new ← S_c^d
    Using Algorithm 2 to get all partial solutions.
    Calculate f(X_winner^new) using Eq (1).
    if f(X_winner^new) < f(X_winner^curr) then
        S_c ← S_c^new
        if f(X_winner^new) < f(X^best) then
            S_c^best ← S_c^new
        end if
    else
        generate a random number rand ∈ [0,1],
        if exp(-(f(X_winner^new) - f(X_winner^curr))/T) > rand then
            S_c ← S_c^new
        end if
    end if
    T = T * ζ
end for
Update ω_p , π_p.
```

3.2.6. Learning phase

In order to improve the search efficiency and speed up the convergence, an inter-team learning method based on crossover operator theory [46,47] is designed in the learning phase of the HVPL. The main content is that the individuals generated by the above search strategies are ranked from largest to smallest according to the value of the objective function, and the top three individuals are used as learning objects for other teams. The remaining h teams randomly select some players to learn from any one of the learning objects. This learning phase extends the effective solution space and makes the individuals evolve towards the optimal solution with high probability. The specific steps are provided below.

(a) Arrange the teams according to the order of the objective function values from smallest to largest,
(b) Select the top three teams as learning objects, denoted by $rank_1, rank_2, rank_3$ respectively,
(c) For the remaining h teams in the league, each team randomly selects the object $rank_i$ to be learned,
(d) In order to make each team learn effectively, each team randomly selects m players to learn skills from the selected learning object,

(e) After learning, the strength of the team is re-evaluated.

In this study, the learning phase of the original VPL was redesigned to solve the combinatorial optimization problem. This phase integrates the population of individuals updated using different strategies and directs the population to the space where better solutions exist. This search strategy was redesigned to not only speed up the convergence, but also improve the quality of the solutions by avoiding the original learning phase from making the algorithm fall into a local optimum [23].

Algorithm 7 Learning phase

$label_1, label_2, label_3$ denote the front, middle and back sections of the team respectively.
Ranking the individuals in the population by the value of the objective function from smallest to largest.
Take the top three individuals, denoted by $rank_1, rank_2, rank_3$ respectively.
for j:= 1 to h **do**
  $rank_i \leftarrow$ team j randomly selects the learning object ,
  $I_1, I_2 \leftarrow$ randomly select two numbers, I_1 and I_2, as breakpoints for the team j segments,
  $label_k \leftarrow$ determine which part of team j is assigned to study,
  if $label_1 \leftarrow label_k$ **then** // $label_2$ and $label_3$ are similar to $label_1$
    for l:= 1 to $S_c^{rank_i}$ **do**
      if l not in the middele and back parts of S_c^j **then**
        add l in the front part of S_c^j,
      end if
    end for
  end if
  Using Algorithm 2 to get all partial solutions.
  Calculate $f(X_{winner}^{new})$ using Eq (1).
end for

3.2.7. Elimination phase

When a season is over, teams with poor power level would be eliminated. From the algorithmic point of view, the elimination phase is a reconfiguration of the population that speeds up the convergence of the HVPL. After $N\delta$ weaker (poor solution quality) teams are eliminated from the league, N denotes the number of individuals in the population and δ denotes the elimination rate. $N\delta$ new teams of the same strength level as the teams that remained join the league. The process of forming new teams is as follows: first, a team j is randomly selected from $N(1-\delta)$ teams, then two positions are randomly selected from the S_c^j of that team j, and finally the customers of the two positions are switched to generate new team. The elimination phase accelerates the convergence of the algorithm while increasing the diversity of the population. The pseudo-code of this phase is shown in Algorithm 8.

Algorithm 8 Elimination phase

Remove $N\delta$ worst teams of league.
for k:= 1 to $N\delta$ **do**
  $team_j \leftarrow$ select randomly from current available teams $(N - N\delta)$,
  A and $B \leftarrow$ select two positions randomly from S_c^j,
  swap the customers of positions A and B to form a new S_c^{new},
  using Algorithm 2 to get all partial solutions,
  calculate $f(X^{new})$ using Eq (1),
  add the new team to league.
end for

4. Experimental results

The HVPL is coded in Python and computed using an Intel Core i7-6700 CPU/3.4GH on Ubuntu 20.04. In the following subsections, we present benchmark instances, settings of important parameters in the algorithm, and numerically compare the HVPL with other methods.

4.1. Benchmark instances

In order to evaluate and analyze the optimization ability of the algorithm, 35 test functions were used for experimentation. Among them, there are 5 unimodal functions with variable dimensions ($F1 \sim F5$), 5 unimodal functions with fixed dimensions ($F6 \sim F10$), 5 multimodal benchmark functions with variable dimensions ($F11 \sim F15$), and 5 multimodal functions with fixed dimensions ($F16 \sim F20$). In addition, 15 CEC-BC-2017 benchmark functions were also incorporated ($F21 \sim F35$). The detailed information of the benchmarks can be found in Table 7.

After reviewing related studies, there is no standard dataset used in the previous literature to validate the performance of the algorithms in solving LRPSPDTW problem. Therefore, 60 instances were randomly selected from the datasets proposed by Solomon [48] to generate the datasets for this paper. The datasets of Solomon are available at http://w.cba.neu.edu/~msolomon/home.htm, which can be classified into three categories according to the distribution of geographic data: clustered distribution C, random distribution R, and mixed distribution RC. More specifically, C1, R1 and RC1 have a short scheduling range, allowing only a few customers on each route while C2, R2 and RC2 have a longer scheduling range, allowing many customers to be served by the same vehicle. The number of candidate depots and the customer pickup requirements were added to the instances of this paper. Each instance has 20, 50 or 100 customers and 3, 5 or 10 candidate depots. For the representation of each instance, we used a string, e.g., R1-104-10d-100c, indicating that the instance numbered R1-104 has 10 candidate depots and 100 customers. To further demonstrate the effectiveness of the method, we validated it using six instances in the NEO Research Group database like pr-01-4d-47c, with the number of customers in the instances ranging from 47 to 287 and the number of candidate depots being 4 or 5, which is located at https://www.bernabe.dorronsoro.es/vrp/.

4.2. Parameter settings

The parameter settings of the HVPL algorithm are described as follows: for the parameter settings, previous studies in the relevant literature were referred. The values of the parameters σ_1, σ_2, σ_3 and ζ were taken from Ropke et al. [49], T_0 and ρ were taken from [50]. In addition, the population size N is set to 10 in this paper. However, there are several parameters in the proposed algorithm that must be tuned, so the Taguchi method proposed by Genichi Taguchi [51] was used. This method can evaluate more factors affecting the experimental results by fewer parameter combinations and obtain stable and less volatile parameter combinations. The factors affecting the experimental results in static Taguchi experiments can be divided into controllable factors (parameters) and noise factors (random errors). The influence of noise on the experimental results is reduced by adjusting the level of controllable factors. The stability of the experimental results is evaluated using the signal-to-noise (S/N) ratio [52], the larger the value, the better the quality, and the best parameter value for each factor is selected based on its maximum S/N ratio. Therefore, this method is one of the most effective methods to study the influence of parameters on algorithm performance [53].

The HVPL has five parameters (factors) that need to be determined to be optimal, each with four levels (Table 8), and each combination is run five times. If the experiment is designed using the full factorial method, it needs to be run $4^5 * 5 = 5120$ times. In contrast, the Taguchi method produced only an orthogonal array (L_{16}) with 16 experiments and used six instances from different data sets for the experiments, which are shown in Table 9.

The results in Table 9 were converted to signal-to-noise ratios (S/N) shown in Table 10, which lists the S/N ratios for the five parameters, with the maximum value of each parameter shown in bold. Tables 8 and 10 are used together to determine the best combination of these four parameters as: $\alpha = 3$, $\beta = 4$, $\gamma = 5$, $\lambda = 40\%$, $\delta = 10\%$.

Table 7
Description of the benchmarks.

Function	*Range*	*Dim*	F_{min}
F1 - Sphere	[−100, 100]	50	0
F2 - Quartic Noise	[−1.28,1.28]	50	0
F3 - Powell Singular	[−4, 5]	50	0
F4 - Zakharov	[−5, 10]	50	0
F5 - Schwefel's 2.23	[−10, 10]	50	0
F6 - Three-Hump Camel	[−5, 5]	2	0
F7 - Brent	[−10, 10]	2	0
F8 - Booth	[−10, 10]	2	0
F9 - Leon	[−1.2, 1.2]	2	0
F10 - Matyas	[−10, 10]	2	0
F11 - Schwefel's 2.26	[−500, 500]	50	0
F12 - Rastrigin	[−5.12, 5.12]	50	0
F13 - Periodic	[−10, 10]	50	0.9
F14 - Ackley	[−32, 32]	50	0
F15 - Alpine N. 1	[−10, 10]	50	0
F16 - Branin RCOS	[−5, 10]	2	0.3978873
F17 - Cross-in-tray	[−10, 10]	2	−2.06261218
F18 - Hartman 6	[0, 1]	6	−3.32237
F19 - Bird	[-2π, 2π]	2	−106.7645
F20 - Camel Six Hump	[−5, 5]	2	−1.0316
F21 - Shifted and Rotated Bent Cigar Function	[−100, 100]	10	100
F22 - Shifted and Rotated Rosenbrock's Function	[−100, 100]	10	300
F23 - Shifted and Rotated Rastrigin's Function	[−100, 100]	10	300
F24 - Shifted and Rotated Expanded Scaffer's $F6$ Function	[−100, 100]	10	400
F25 - Shifted and Rotated Lunacek Bi-Rastrigin Function	[−100, 100]	10	600
F26 - Shifted and Rotated Non-Continuous Rastrigin's Function	[−100, 100]	10	700
F27 - Shifted and Rotated Levy Function	[−100, 100]	10	800
F28 - Shifted and Rotated Schwefel's Function	[−100, 100]	10	900
F29 - Hybrid Function 1 (N = 3)	[−100, 100]	10	1000
F30 - Hybrid Function 3 (N = 3)	[−100, 100]	10	1200
F31 - Hybrid Function 4 (N = 4)	[−100, 100]	10	1300
F32 - Hybrid Function 5 (N = 4)	[−100, 100]	10	1400
F33 - Composite Function 2 (N = 3)	[−100, 100]	10	2100
F34 - Composite Function 6 (N = 5)	[−100, 100]	10	2500
F35 - Composite Function 8 (N = 6)	[−100, 100]	10	2700

Table 8
The HVPL's parameters.

Parameter	Level 1	Level 2	Level 3	Level 4
α	1	2	3	4
β	1	2	3	4
γ	4	5	6	7
λ	10%	20%	30%	40%
δ	5%	10%	15%	20%

4.3. Performance analysis of the operators

In the HVPL algorithm, the repositioning strategy (RS) and the substitution strategy (S2) were used together as local search operators for the ACO. To analyze the performance of these operators, a series of experiments were conducted using six instances from different datasets. In Table 11, the column labeled " All" gives the results obtained with equal probability using all local search operators. The other columns show the results obtained using the operators alone. The result of each instance in the experiment is the average of 10 runs. The smaller the value of the objective function for the same instance, the more significant the role played by the operator in the algorithm. From the table, we can see that S2 has a stronger impact on the performance of the HVPL than RS, and comparing the column labeled "ALL" with the other two columns, we can see that both RS and S2 operators play a synergistic role in the algorithm.

4.4. Comparison of the results from test functions

In order to evaluate the exploitation and exploration capabilities of the algorithm, the HVPL algorithm was compared with the Heap-based optimizer (HBO) [54], Political optimizer (PO) [55], Gradient-based optimizer (GBO) [56], Marine predators algorithm (MPA) [57], Manta ray foraging optimization (MRFO) [58], Forensic-based investigation (FBI) algorithm [59], and Volleyball Premier League (VPL) Algorithm [22]. To obtain optimal results in the benchmark functions experiment, each algorithm utilized the recommended parameter values from the respective papers. To ensure fairness in the comparison, all algorithms had a population size of 40, and the iteration of the algorithm was concluded once the number of function evaluations reached 40,000. Furthermore, each algorithm was executed independently 25 times for each benchmark function, and the average value and standard deviation of the results are presented. It is worth noting that, except for VPL, the other comparative algorithms were recommended by Alorf after analyzing the optimization capabilities of 26 recently developed meta-heuristic algorithms using test functions and engineering optimizations [60]. To measure the overall performance of each algorithm on each set of test functions and determine if there are significant performance differences, we used the Friedman mean rank to rank all algorithms, which is a non-parametric statistical test [61].

We analyzed unimodal functions ($F1 \sim F10$) to measure the exploitation capability of the algorithm, as these test functions have only one global optimum value and no local optimum values. $F1 \sim F10$ in Table 12 show the comparison results of the algorithms for the unimodal benchmark functions, including the average, standard deviation, and rank of each algorithm relative to each function. It can be seen that HVPL performs the best. The Friedman test of unimodal functions gives the mean rank of all algorithms as shown in Fig. 5. The smallest bar or the lowest value on the bar graph represents the best average rank. The results show that HVPL ranks first. In addition, to test the convergence capability of HVPL in practice, we plotted the convergence curve of HVPL for unimodal functions. For different unimodal functions, HVPL has two convergence modes. One is gradually decreasing during the solving process, which is evident in Fig. 8(a). The other is that the algorithm's speed tends to accelerate

Table 9
The results of the orthogonal array $L_{16}(4^5)$.

Orthogonal array						Instances					
#	α	β	γ	λ	δ	C1-102-5d-30c	C2-208-3d-20c	R1-101-5d-30c	R2-208-5d-40c	RC1-103-5d-30c	RC2-206-3d-20c
1	1	1	4	10%	5%	294.447	196.295	396.984	674.541	297.351	322.833
2	1	2	5	20%	10%	280.245	183.025	388.683	657.006	284.024	312.543
3	1	3	6	30%	15%	287.244	193.573	395.023	668.122	292.642	319.854
4	1	4	7	40%	20%	279.562	182.054	386.328	651.036	280.895	311.354
5	2	1	5	30%	20%	285.652	187.428	392.989	667.125	290.624	316.732
6	2	2	4	40%	15%	284.325	182.657	387.088	662.411	284.732	310.042
7	2	3	7	10%	10%	291.323	193.471	394.987	667.430	292.173	318.843
8	2	4	6	20%	5%	289.234	188.302	390.901	666.872	289.304	317.043
9	3	1	6	40%	10%	283.254	182.045	383.539	651.844	279.104	307.243
10	3	2	7	30%	5%	290.326	198.350	395.671	667.282	296.985	321.853
11	3	3	4	20%	20%	287.842	189.054	389.923	668.983	287.355	316.512
12	3	4	5	10%	15%	283.355	182.605	385.892	652.629	280.392	307.841
13	4	1	7	20%	15%	285.234	186.350	387.935	666.502	290.850	316.517
14	4	2	6	10%	20%	290.345	195.022	397.035	674.002	296.147	323.105
15	4	3	5	40%	5%	280.756	183.624	385.890	652.013	281.154	309.624
16	4	4	4	30%	10%	280.564	181.602	387.625	659.861	285.341	310.630

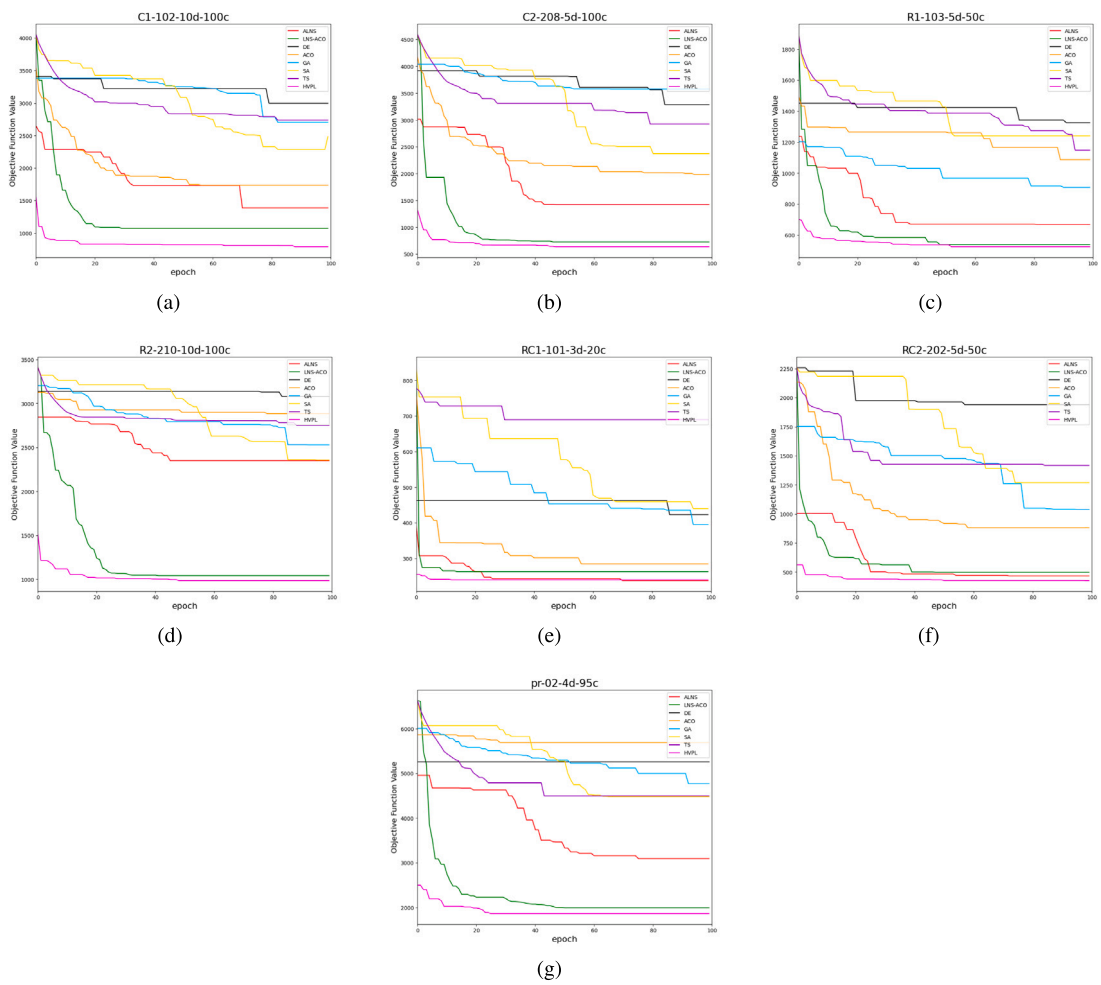


Fig. 4. The convergence curves of HVPL and other comparison algorithms on seven instances.

with increasing iterations increases , and then drops sharply, as shown in Fig. 8(b).

Multimodal functions with many local optima are used to evaluate the exploration ability of algorithms. $F11 \sim F20$ in Table 12 show the comparison results of the algorithms with multimodal benchmark functions. Except for $F14$, HVPL found the optimal values, indicating that HVPL has good development capability. The average ranks of all algorithms given by the Friedman test for multimodal functions are shown in Fig. 6. The results show that HVPL outperforms other compared algorithms. In addition, for different multimodal functions, HVPL has three convergence modes. One mode is to maintain fast growth in convergence speed from the beginning without falling into local optima, as shown in Fig. 8(c). The second mode is to gradually decrease the exploitation speed during the solving process, which is in Fig. 8(e). The third mode is that the algorithm's convergence speed is initially fast, then sharply decreases and falls into local optima, and

Table 10
The S/N ratios of the parameters.

Level	α	β	γ	λ	δ
1	−50.354	−50.357	−50.346	−50.438	−50.418
2	−50.365	−50.383	−50.234	−50.328	−50.261
3	−50.307	−50.369	−50.381	−50.395	−50.301
4	−50.316	−50.231	−50.379	−50.180	−50.361
Delta	0.058	0.152	0.147	0.258	0.157
Rank	5	3	4	1	2

Table 11
Performance comparison of the ACO's local search operators.

Instance	RS	S2	All
C1-102-5d-30c	364.717	317.782	294.447
C2-208-3d-20c	213.756	196.295	183.101
R1-101-5d-30c	485.052	400.236	388.998
R2-208-5d-40c	984.676	619.806	612.972
RC1-103-5d-30c	308.725	279.105	279.105
RC2-206-3d-20c	324.212	317.833	307.488

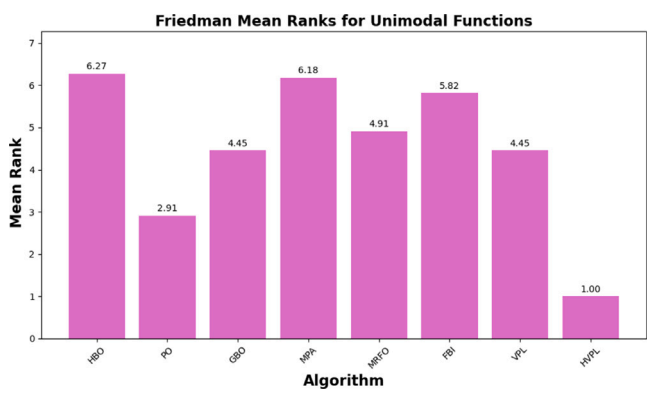


Fig. 5. Friedman mean ranks of all algorithms for unimodal benchmark functions.

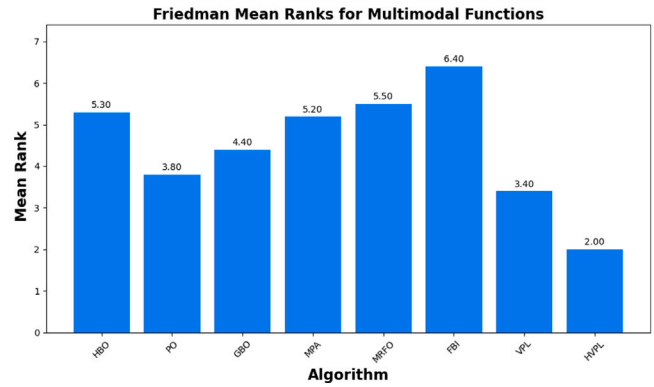


Fig. 6. Friedman mean ranks of all algorithms for multimodal benchmark functions.

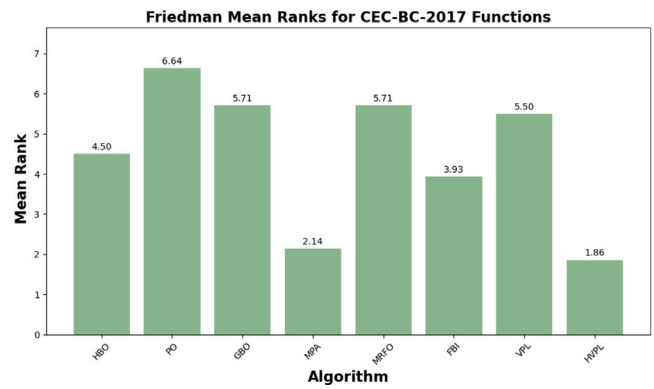


Fig. 7. Friedman mean ranks of all algorithms for CEC-BC-2017 benchmark functions.

then jumps out of the local optima and slowly converges to the global optimum, as shown in Fig. 8(d). It can be seen that HVPL has powerful ability to explore the search space.

In order to quantify the optimization capability of HVPL for equilibrium exploration and exploitation, the CEC-BC-2017 benchmark functions are used to evaluate this ability. These functions include unimodal, multimodal, hybrid, and composite functions, which have highly challenging terrains. The algorithms results of the CEC-BC-2017 functions are shown in Table 12, from $F21$ to $F35$. Except for $F21$ and $F28$, HVPL can obtain good solutions on most of the functions. In addition, Friedman test was used to statistically rank these algorithms, and the results are shown in Fig. 7. The results show that HVPL ranks first, MPA ranks second, and the difference in ranking between HVPL and MPA is not significant. Due to the fact that the algorithm proposed in this paper is mainly used to solve combinatorial optimization problems similar to LRPSPDTW, according to the "no free lunch" (NFL) theorems [62], although HVPL has a relatively good balance between exploration and exploitation, it is not significantly better than MPA. Fig. 8(f) show the convergence curves of HVPL for the CEC-BC-2017 functions, and due to the complex region of the functions, the HVPL algorithm gradually converges to the optimal or suboptimal solutions.

To provide a more intuitive demonstration of the exploration and exploitation process of the HVPL algorithm in finding the optimal solution, Fig. 9 presents images of the proposed algorithm searching the solution space of both unimodal and multimodal functions in 2D. From Fig. 9(a), it can be observed that as the number of iterations increases, the individuals gradually converge to a promising region in the search for the optimal solution of the unimodal function $F9$. This indicates that HVPL has good development capability. Fig. 9(b) shows the search process of the HVPL algorithm in the solution space of the multimodal function $F19$. With a few iterations, several clusters can be seen in the solution space. As the number of iterations accumulates, the population gradually explores and locates the region containing the global optimal solution. This demonstrates the strong exploration ability of the proposed algorithm.

4.5. Comparison of the results from LRPSPDTW instances

To test the performance of HVPL, we tested it using 66 instances from seven datasets with different distributions and scheduling ranges. To evaluate the performance of HVPL by comparing it with several well-established and widely cited algorithms from the existing literature. These algorithms include the adaptive large neighborhood search (ALNS) [63], LNS-ACO [64], differential evolution(DE) [65], ant colony optimization(ACO) [66], genetic algorithm(GA) [17], simulated annealing(SA) [15] and tabu search(TS) [12]. These methods[1,2] were also coded in Python, their parameter values were similarly fine-tuned using the Taguchi method. The best possible parameter values are reported in Appendix. Each method was run 10 times on each instance, and the maximum number of iterations per run was set to 100 for all methods except the HVPL. Since the HVPL uses SRR, which can be regarded as a strategy execution rule, the proposed algorithm executes its search strategies for individuals in the population $(N-1)$ times at each iteration. The population size N of the algorithm is set to 10. For a fair comparison with other methods, the number of iterations of the proposed algorithm is set to 11. According to the SRR league schedule, each week is considered as one iteration in this paper, then the

[1] https://github.com/ShuoSun1994/HVPL
[2] https://github.com/search?q=hybrid+LNS

Table 12
The HVPL and other algorithms for the benchmark.

	HBO	PO	GBO	MPA	MRFO	FBI	VPL	HVPL
Unimodal								
$F1$								
Avg	8.8E−11	0	9E−262	1.1E−45	0	4E−231	0	0
Std	8.5E−11	0	0	4.8E−45	0	0	0	0
Rank	8	**1**	5	7	**1**	6	**1**	**1**
$F2$								
Avg	0.03693	0.00022	0.00029	0.0006	0.00011	8.1E−05	0.00024	0
Std	0.00729	0.00021	0.0002	0.00032	9.1E−05	6.9E−05	0.00053	0
Rank	8	4	6	7	3	2	5	**1**
$F3$								
Avg	0.0999	0	3E−250	8.7E−19	0	2E−228	0	0
Std	0.06422	0	0	4.3E−18	0	0	0	0
Rank	8	**1**	6	7	**1**	5	**1**	**1**
$F4$								
Avg	387.014	0	2E−191	3.8E−05	0	1E−98	0	0
Std	63.2533	0	0	0	0	5.1E−98	0	0
Rank	8	**1**	5	7	**1**	6	**1**	**1**
$F5$								
Avg	1E−17	0	0	1E−193	0	0	0	0
Std	2.1E−17	0	0	0	0	0	0	0
Rank	8	**1**	**1**	7	**1**	**1**	**1**	**1**
$F6$								
Avg	6E−197	0	0	3E−147	0	0	4.339E−10	0
Std	0	0	0	1E−146	0	0	2.6E−03	0
Rank	6	**1**	**1**	7	**1**	**1**	8	**1**
$F7$								
Avg	1.4E−87	1.4E−87	1.4E−87	1.4E−87	1.4E−87	3E−23	1.38E−87	1.38E−87
Std	5E−103	5E−103	5E−103	5E−103	5E−103	6.8E−23	4.6E−105	0
Rank	3	3	3	3	3	8	2	**1**
$F8$								
Avg	0	0	0	0	0	0	0	0
Std	0	0	0	0	0	0	0	0
Rank	**1**	**1**	**1**	**1**	**1**	**1**	**1**	**1**
$F9$								
Avg	3.9E−11	0	0	0	9.4E−12	2E−16	1.12E−8	0
Std	1.5E−10	0	0	0	3.6E−11	6E−16	1.2E−4	0
Rank	7	**1**	**1**	**1**	6	5	8	**1**
$F10$								
Avg	1.1E−70	0	0	2E−117	0	0	5.19E−11	0
Std	5.3E−70	0	0	1E−116	0	0	6.2E−56	0
Rank	7	**1**	**1**	6	**1**	**1**	8	**1**
Multimodal								
$F11$								
Avg	23.9151	9.47508	116.764	103.424	159.737	161.698	0	0
Std	7.52135	32.7941	21.8015	12.361	19.3397	10.2596	0	0
Rank	4	3	6	5	7	8	**1**	**1**
$F12$								
Avg	20.039	7.95967	0	0	401.126	478.642	0	0
Std	3.36203	18.614	0	0	0	0	0	0
Rank	6	5	**1**	**1**	7	8	**1**	**1**
$F13$								
Avg	4.22752	0.952	0.9	1.00868	0.9	2.18983	0.9	0.9
Std	0.96784	0.05099	3.4E−16	0.00501	3.4E−16	1.00824	0	0
Rank	8	5	2	6	2	7	**1**	**1**
$F14$								
Avg	1.9E−06	−9E−16	−9E−16	2.5E−15	−9E−16	−9E−16	4.4E−16	4.4E−16
Std	1.4E−06	0	0	7.1E−16	0	0	1.3E−126	0
Rank	8	3	3	7	3	3	2	**1**
$F15$								
Avg	1.4E−05	0	6E−135	8.9E−27	5E−212	1E−119	0	0
Std	6.2E−05	0	2E−134	2.1E−26	0	9E−120	0	0
Rank	8	**1**	5	7	4	6	**1**	**1**
$F16$								
Avg	0.39789	0.39789	0.39789	0.39789	0.39789	0.39789	0.39789	0.39789
Std	0	0	0	0	0	0	0	0
Rank	**1**	**1**	**1**	6	**1**	**1**	**1**	**1**

(*continued on next page*)

Table 12 (*continued*).

	HBO	PO	GBO	MPA	MRFO	FBI	VPL	HVPL
$F17$								
Avg	−2.0626	−2.0626	−2.0626	−2.0626	−2.0626	−2.0626	−2.0626	−2.0626
Std	9.1E−16	9.1E−16	9.1E−16	9.1E−16	9.1E−16	9.1E−16	0	0
Rank	3	3	3	3	3	3	**1**	**1**
$F18$								
Avg	−3.322	−3.3125	−3.2649	−3.322	−3.2744	−3.322	−3.322	−3.322
Std	4.5E−16	0.03292	0.06062	1.5E−15	0.05945	5E−06	1.2E−126	0
Rank	3	6	8	4	7	5	2	**1**
$F19$								
Avg	−106.76	−106.76	−106.76	−106.76	−106.76	−106.76	−106.76	−106.76
Std	2.9E−15	1.8E−14	8.2E−15	1.3E−14	2.2E−14	8.7E−15	1.8E−19	0
Rank	3	7	4	6	8	5	2	**1**
$F20$								
Avg	−1.0316	−1.0316	−1.0316	−1.0316	−1.0316	−1.0316	−1.0316	−1.0316
Std	6.7E−16	6.7E−16	6.8E−16	6.4E−16	6.8E−16	6.8E−16	6.7E−16	0
Rank	3	3	4	2	2	2	3	**1**
CEC-BC-2017								
$F21$								
Avg	490.1832	6284.225	1928.807	100.0001	2825.001	300.0749	1.03E+09	9.2E+07
Std	406.4005	4490.46	1822.998	9.95E−05	3313.91	3183.31	2.3E+09	0
Rank	3	6	4	**1**	5	2	8	7
$F22$								
Avg	301.9884	496.4448	300	300	300	300.0031	387.7967	300
Std	3.133711	299.0156	6.86E−14	1.48E−09	1.2E−11	0.008194	177.571	1.02E−28
Rank	6	8	2	4	3	5	7	**1**
$F23$								
Avg	405.0895	405.6096	400.0867	400	401.3236	400.4197	410.9463	400
Std	0.899104	400.103	282.038	317.633	0.683112	0.262025	177.571	0.053
Rank	6	7	3	2	5	4	8	**1**
$F24$								
Avg	512.3064	512.0532	527.7405	508.7288	521.4911	509.9561	524.1564	500.4983
Std	3.662879	6.567678	11.59847	3.258824	8.324389	2.011018	6.589421	2.675225
Rank	5	4	8	2	6	3	7	**1**
$F25$								
Avg	600	609.5841	600.0696	600.0011	601.9358	600.0002	627.9878	600.0259
Std	2.84E−07	16.22112	0.161223	0.003836	3.78755	0.000315	17.7578	0.007862
Rank	**1**	7	5	3	6	2	8	4
$F26$								
Avg	724.9839	719.0217	737.6339	721.2798	746.611	727.7597	712.9665	708.2637
Std	3.797462	3.943396	10.34231	4.1783973	15.5968	3.580526	1.5E−06	4E−124
Rank	5	3	7	4	8	6	2	**1**
$F27$								
Avg	812.2694	834.1004	822.3268	808.477	819.5352	809.4219	837.4248	800.1994
Std	4.756199	4.585033	7.059269	3.326294	7.487354	3.178191	6E−24	1.3E−120
Rank	4	7	6	2	5	3	8	**1**
$F28$								
Avg	900	1562.54	907.4759	900	912.7819	900.1198	1709.0856	1162.17
Std	0	334.5499	25.07853	3.1E−07	26.76818	0.202605	0.4732	6.5E−04
Rank	**1**	7	4	2	5	3	8	6
$F29$								
Avg	1700.123	1863.125	1912.783	1365.543	1830.829	1469.379	1042.570	1004.432
Std	143.946	293.370	332.139	155.658	342.168	109.184	61.287	1.527
Rank	5	7	8	3	6	4	2	**1**
$F30$								
Avg	97 816.42	28 195.66	19 590.56	1212.518	14 732.74	12 480.87	14 047.16	3206.05
Std	108 255.4	24 606.49	16 558.59	26.32236	12 809.53	6983.591	13.258	56.541
Rank	8	7	6	**1**	5	3	4	2
$F31$								
Avg	2222.283	12 374.34	1583.617	1304.898	2938.143	4312.033	1405.105	1363.613
Std	1037.519	14 057.79	275.8515	2.077764	1391.718	2488.008	10.424	1.10624
Rank	5	8	4	**1**	6	7	3	2
$F32$								
Avg	1461.153	1559.144	1493.959	1403.463	1470.673	1465.031	1463.625	1406.933
Std	58.61203	57.81456	38.46783	2.012688	21.10761	22.43568	11.2418	18.96991
Rank	3	8	7	**1**	6	5	4	2
$F33$								
Avg	2263.182	2311.066	2273.922	2200	2231.547	2205.099	2222.328	2200.5671
Std	55.54827	9.882577	61.37039	1.13E−05	52.75725	22.81764	19.55521	0.112105
Rank	6	8	7	**1**	5	3	4	2

(*continued on next page*)

Table 12 (*continued*).

	HBO	PO	GBO	MPA	MRFO	FBI	VPL	HVPL
*F*34								
Avg	2918.958	2935.728	2927.37	2885.835	2922.728	2877.411	2890.402	2703.066
Std	22.60715	19.42267	23.46776	59.54181	23.58915	67.6373	5.7E−14	9.03324
Rank	5	8	7	3	6	2	4	**1**
*F*35								
Avg	3091.521	3091.878	3102.468	3089.002	3104.352	3094.697	3172.844	3000.010
Std	2.186673	2.510503	17.47848	0.420016	14.79162	2.986285	4.235	6.62E−12
Rank	3	4	6	2	7	5	8	**1**

* All algorithms are trimmed in bold when ranked first on the corresponding test function.
* The gray background data indicates that HVPL ranked first or tied for first place in the results on the corresponding test function.

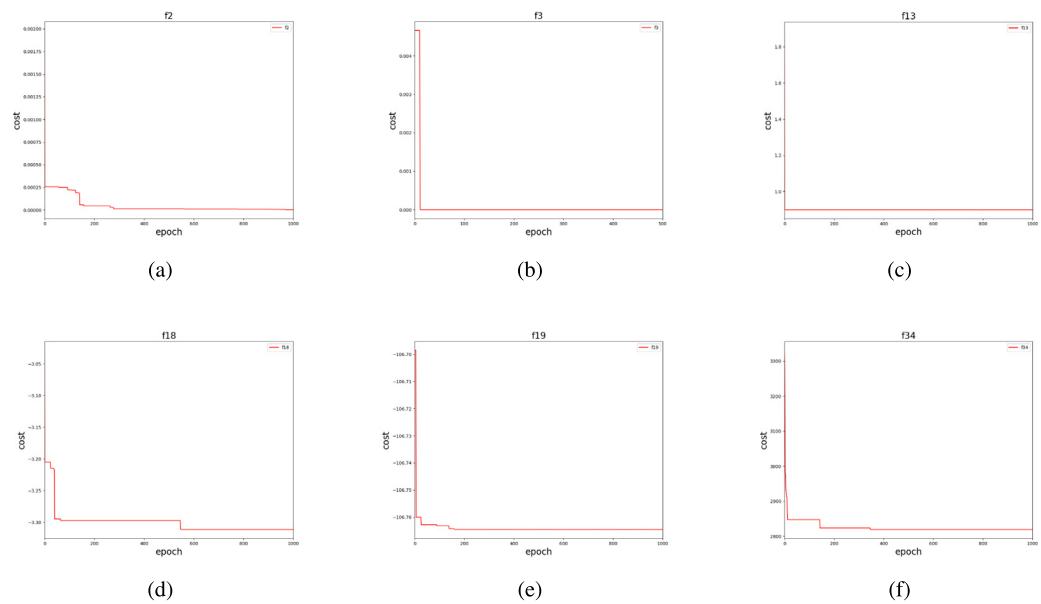


Fig. 8. The convergence curves of the HVPL for selected functions.

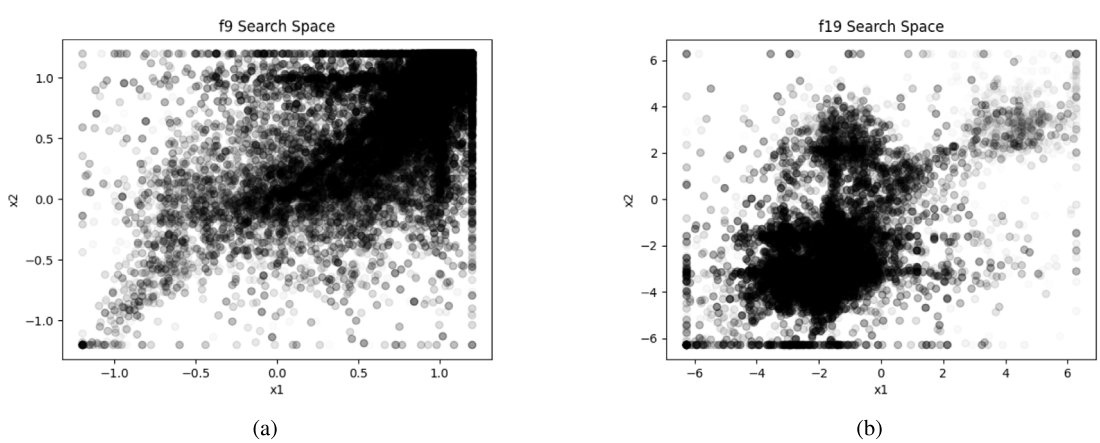


Fig. 9. Search space for unimodal function F9 and multimodal function F19.

algorithm is iterated 99 times in total, but the learning and elimination phases of the method are executed only 11 times. The results obtained are listed in Table 13 - Table 19. In the table, "Best" denotes the best result among all runs for each instance. "Avg". is the average of the results obtained from all runs in each instance. "CPU" is the time, in seconds, that the algorithm was run on each instance.

Tables 13 and 14 show the results of the methods on 20 instances which belong to the datasets C1 and C2 with the clustered distribution. In terms of "Best", the HVPL achieved the best result in 18 instances, only in instances C1-109-5d-20c and C2-208-10d-100c which is worse than the LNS-ACO. Comparing the "Avg"., the HVPL achieved the best value in all instances except C2-208-10d-100c. In the other methods, the LNS-ACO achieved better results.

Tables 15 and 16 list the results of the methods on datasets R1 and R2 with random distribution, containing 24 instances. In terms of "Best", the HVPL achieved the best results in 21 instances compared to the other methods, and only on R1-111-5d-50c, R2-202-3d-20c and R2-207-10d-100c the results are second to the LNS-ACO. Comparing the "Avg"., the HVPL achieved the best results in all instances except R2-202-3d-20c. The best performance among the other methods is the LNS-ACO, which is second only to the HVPL.

Table 13
The HVPL and other algorithms for dataset C1.

	ALNS	DE	ACO	GA	SA	TS	LNS-ACO	HVPL
C1-101-3d-20c								
Best	213.705	358.975	286.480	275.954	420.634	472.766	**177.571**	**177.571**
Avg.	213.705	358.975	289.357	278.347	420.634	475.075	**177.571**	**177.571**
C1-109-5d-20c								
Best	217.863	289.653	253.642	272.875	434.269	578.953	**203.896**	204.495
Avg.	217.863	289.653	264.832	275.964	434.269	581.940	206.586	**204.495**
C1-103-5d-50c								
Best	594.305	1564.025	752.407	985.380	863.738	1404.084	482.584	**390.394**
Avg.	598.368	1795.189	789.389	1097.094	966.941	1404.084	542.437	**390.395**
C1-107-5d-50c								
Best	763.466	1192.997	762.798	1404.093	1285.251	1168.946	427.008	**403.728**
Avg.	772.406	1203.207	775.994	1525.004	1320.961	1323.706	437.805	**410.697**
C1-102-5d-100c								
Best	1557.618	2607.082	1528.713	2691.140	2652.922	2340.771	871.924	**774.529**
Avg.	1561.906	2700.991	1582.591	2811.291	2695.615	2718.162	901.462	**774.529**
C1-102-10d-100c								
Best	1383.732	2950.954	1732.138	2689.002	2276.021	2726.943	1023.309	**791.169**
Avg.	1404.008	3000.603	1812.809	2850.911	2369.082	2789.490	1089.318	**807.469**
C1-105-5d-100c								
Best	2912.193	3201.613	2901.301	3472.051	3068.196	2761.402	1132.106	**1098.266**
Avg.	2981.207	3321.096	3000.938	3591.521	3114.516	2801.278	1137.184	**1098.266**
C1-105-10d-100c								
Best	2373.252	2672.967	2721.901	2952.631	2506.935	2608.417	1036.063	**1031.708**
Avg.	2407.642	2702.053	2742.908	3001.582	2583.276	2732.900	1043.725	**1031.708**
C1-108-5d-100c								
Best	1362.863	2945.732	1836.001	2654.049	2826.893	2284.205	831.931	**823.174**
Avg.	1379.004	3120.831	1872.762	2730.816	2830.921	2352.688	858.331	**828.357**
C1-108-10d-100c								
Best	1283.478	2773.536	1721.432	2592.325	2565.125	2473.894	926.046	**759.819**
Avg.	1361.676	2905.329	1765.586	2733.456	2569.161	2557.534	971.005	**760.649**

Table 14
The HVPL and other algorithms for dataset C2.

	ALNS	DE	ACO	GA	SA	TS	LNS-ACO	HVPL
C2-201-3d-20c								
Best	214.623	359.921	272.962	293.729	373.625	428.825	197.830	**196.372**
Avg.	214.623	359.921	272.962	272.962	375.490	428.825	197.830	**196.372**
C2-207-5d-20c								
Best	213.662	309.603	251.632	238.098	304.111	427.638	188.836	**187.807**
Avg.	213.662	309.603	252.056	238.098	305.701	427.638	188.836	**187.807**
C2-203-5d-50c								
Best	652.297	1492.642	1023.384	1040.857	1229.493	1373.317	488.600	**364.732**
Avg.	663.735	1527.251	1052.821	1093.705	1334.182	1432.007	491.662	**364.732**
C2-206-5d-50c								
Best	629.092	1043.992	868.582	1063.721	1385.386	1126.040	408.942	**390.161**
Avg.	641.084	1125.002	873.942	1122.942	1501.742	1173.030	417.702	**390.161**
C2-202-5d-100c								
Best	1293.625	3356.903	2691.022	3424.025	2993.946	3293.261	762.621	**705.086**
Avg.	1326.027	3524.665	2727.025	356.925	3103.627	3426.730	782.062	**705.086**
C2-202-10d-100c								
Best	1066.021	3391.724	2053.621	2951.863	2853.925	2732.824	722.141	**660.924**
Avg.	1092.993	3398.093	2096.063	2951.863	2862.618	2745.040	752.809	**667.455**
C2-205-5d-100c								
Best	1673.227	3063.052	2227.529	3628.262	2426.752	3142.528	773.824	**667.212**
Avg.	1716.259	3193.572	2267.091	3634.701	2441.427	3142.528	776.317	**667.721**
C2-205-10d-100c								
Best	1094.906	3160.032	1850.926	2918.973	2934.315	2230.922	748.153	**587.099**
Avg.	1123.001	3247.036	1873.363	3024.962	3002.529	2256.720	773.361	**595.346**
C2-208-5d-100c								
Best	1424.722	3285.526	1935.901	3578.474	2374.479	2924.746	725.700	**638.716**
Avg.	1424.722	3285.526	2001.573	3578.474	2426.002	2924.746	727.036	**638.716**
C2-208-10d-100c								
Best	852.582	2962.320	1921.736	2772.737	2427.842	2724.921	**683.620**	689.099
Avg.	858.472	2962.320	1922.672	2772.737	2431.005	2724.921	**689.102**	691.762

Tables 17 and 18 present the results of the methods on 16 instances with mixed distribution. In terms of "Best", the HVPL obtained better results than the other compared algorithms on 14 instances, and only on RC1-101-3d-20c and RC1-103-5d-100c are the results inferior to

Table 15
The HVPL and other algorithms for dataset R1.

	ALNS	DE	ACO	GA	SA	TS	LNS-ACO	HVPL
R1-102-3d-20c								
Best	628.039	527.009	612.557	583.037	579.295	682.523	546.992	**525.983**
Avg.	628.039	527.009	612.557	583.037	579.295	682.523	546.992	**525.983**
R1-112-5d-20c								
Best	477.024	544.053	583.840	522.242	579.226	601.003	463.429	**439.142**
Avg.	477.024	544.053	583.840	522.242	579.902	601.003	463.429	**439.213**
R1-103-5d-50c								
Best	667.220	1323.085	1079.965	902.010	1230.995	1132.526	534.742	**524.706**
Avg.	669.085	1357.690	1088.631	911.747	1262.526	1156.300	540.624	**525.189**
R1-111-5d-50c								
Best	822.269	1201.256	1311.252	1299.444	1194.893	1363.293	**621.452**	627.023
Avg.	831.530	1321.500	1319.266	1318.162	1202.356	1371.554	633.009	**629.569**
R1-101-5d-100c								
Best	2597.006	3115.653	3561.361	3179.605	2700.943	2967.796	1589.005	**1574.469**
Avg.	2603.688	3152.789	3596.966	3225.245	2715.768	2989.099	1601.003	**1579.783**
R1-101-10d-100c								
Best	2762.470	2330.877	2563.575	2321.687	2183.005	2264.464	1442.673	**1426.048**
Avg.	2794.385	2343.523	2571.310	2340.301	2205.300	2310.563	1448.641	**1427.367**
R1-104-5d-100c								
Best	1405.574	2892.340	2729.300	2878.054	2594.805	2564.524	1096.790	**1069.292**
Avg.	1418.501	2922.663	2734.037	2885.036	2603.712	2569.939	1101.004	**1077.683**
R1-104-10d-100c								
Best	1287.001	2289.082	2514.943	2399.704	2481.272	2524.637	936.463	**868.732**
Avg.	1293.112	2305.233	2521.170	2412.241	2501.502	2527.865	942.844	**869.815**
R1-107-5d-100c								
Best	1238.973	2901.100	2662.552	2350.293	2305.222	2336.044	1076.223	**993.075**
Avg.	1284.966	2967.700	2711.408	2382.658	2351.321	2351.763	1082.926	**996.732**
R1-107-10d-100c								
Best	1522.530	2534.492	2301.117	2095.001	2135.356	1952.962	952.531	**940.029**
Avg.	1547.852	2584.006	2394.831	2125.930	2172.681	2021.367	964.610	**942.423**
R1-110-5d-100c								
Best	1372.137	2963.911	2436.792	2703.532	2053.560	2470.012	1262.512	**1026.209**
Avg.	1373.623	2963.911	2511.638	2730.046	2119.260	2470.012	1268.211	**1030.624**
R1-110-10d-100c								
Best	1238.941	2811.930	2247.008	2425.663	2057.855	2005.426	946.060	**904.187**
Avg.	1252.039	2830.026	2259.681	2425.663	2066.100	2104.040	962.016	**909.239**

those of LNS-ACO and ALNS, respectively. LNS-ACO obtained results second to the HVPL on most instances, but there are a few instances where the results are not as good as the HVPL and the ALNS, such as RC2-201-3d-20c and RC2-202-5d-50c. To compare the "Avg". of the methods, the HVPL achieved the best results in these 15 instances. the LNS-ACO took the majority of the results second only to the HVPL.

The pr dataset was used to examine the ability of the proposed algorithm to solve problems with different customer sizes, and there are six instances in this dataset with customer sizes ranging from 47–287. Table 19 lists the results of the algorithms. In terms of "Best", HVPL outperformed other methods in all instances except pr-01-4d-47c. Additionally, in terms of "Avg"., HVPL demonstrated superior performance across all six instances, indicating that the HVPL is more suitable for solving the LRPSDPTW problem. Among the other methods, the LNS-ACO yielded the best results. It can be seen that the performance of the hybrid algorithm is generally higher than that of the single algorithm. From the average of the CPU times in the table, the HVPL takes more time compared to other methods. This is due to the amount of time consumed in constructing feasible solutions using the improved Bellman's algorithm. Fig. 10 presents the routes of the solutions produced by the HVPL in pr-01-4d-47c and pr-06-5d-287c, respectively.

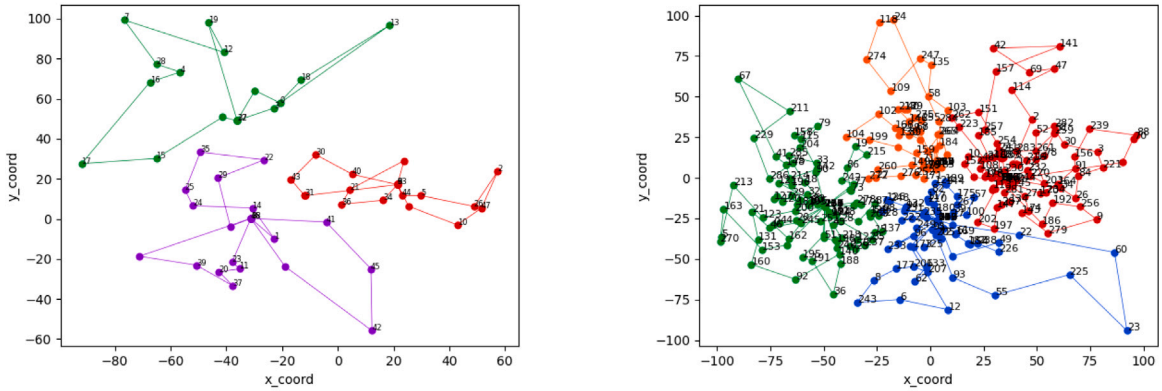
4.6. Comparison of the convergence of the HVPL and other methods

The ALNS, DE, ACO, GA, SA, TS, LNS-ACO and HVPL were run on seven instances from different datasets. The convergence curves of these methods are shown in Fig. 4. In the figure, the convergence curves of the HVPL are highlighted by three features. The first feature is that the HVPL starts the search with better initial solutions than other algorithms, and its initial values can be at most three times better than the compared algorithms on some instances (e.g., Fig. 4(b), (c) and (g)). This is because the initial solutions generated by the Bellman's algorithm [35] are all feasible, so the initial solution quality is higher than that of other methods. The second feature is that the HVPL can still continue to improve if it falls into a local optimum in the first half of the iteration. As shown in Fig. 4(a), the HVPL falls into a local optimum at the 14th iteration, but jumps out of the local optimum at the 63rd iteration and continues to converge until the best solution is obtained at the 87th iteration. This is greatly achieved not only by the ALNS, but also by the use of local search operators RS and S2 with a certain probability. The third feature is that the method eventually converges to a better solution than the other methods. This mainly benefits from the coordinated cooperation of the search strategies, which expands the search space, and the learning and elimination phases guide the search strategies towards high-quality solutions.

4.7. Statistical analysis

In this subsection, we aimed to demonstrate whether the optimal solution obtained by the proposed algorithm differs significantly from other methods. We used Scipy 3[3] to perform Wilcoxon Signed-Rank tests on the HVPL and the other algorithms. Firstly, we randomly selected one instance from each dataset, and then ran the HVPL and other methods 30 times on each selected instance separately to obtain

[3] https://docs.scipy.org/doc/scipy/index.html



(a) (b)

Fig. 10. (a) The routes of pr-01-4d-47c (b)The routes of pr-06-5d-287c.

Table 16
The HVPL and other algorithms for dataset R2.

	ALNS	DE	ACO	GA	SA	TS	LNS-ACO	HVPL
R2-202-3d-20c								
Best	422.821	462.637	416.816	303.021	451.250	600.020	**285.743**	286.820
Avg.	422.821	462.637	417.964	303.021	452.942	600.020	**285.743**	286.940
R2-209-5d-20c								
Best	394.332	551.604	558.084	463.899	512.042	613.001	324.051	**311.853**
Avg.	394.332	551.604	559.262	463.899	515.007	613.001	324.051	**311.853**
R2-203-5d-50c								
Best	1781.008	1976.239	1989.763	1872.446	1823.033	1893.2089	735.930	**725.992**
Avg.	1801.207	1983.002	1997.824	1882.931	1834.991	1924.988	739.083	**725.992**
R2-208-5d-50c								
Best	1398.901	1893.707	1783.385	1684.296	1620.444	1721.205	674.380	**662.939**
Avg.	1452.006	1900.035	1786.449	1702.942	1627.102	1734.082	675.931	**662.939**
R2-201-5d-100c								
Best	2235.788	3324.940	3007.253	3124.043	2796.038	2744.473	1058.836	**990.590**
Avg.	2248.644	3332.004	3017.548	3141.763	2843.185	2761.996	1073.009	**991.000**
R2-201-10d-100c								
Best	1902.052	2723.021	2758.925	2614.036	2425.062	2261.359	1021.662	**966.382**
Avg.	1942.792	2754.933	2769.032	2635.922	2429.003	2272.962	1037.936	**968.072**
R2-204-5d-100c								
Best	3187.712	3601.063	3924.203	3795.731	3199.200	3264.920	1194.842	**1027.318**
Avg.	3210.525	3662.821	3936.825	3823.661	3213.950	3275.207	1203.037	**1028.152**
R2-204-10d-100c								
Best	2279.040	2925.129	2833.525	3273.006	2912.844	2561.268	1056.473	**1007.601**
Avg.	2357.927	2941.832	2871.763	3428.963	2937.251	2590.742	1066.093	**1011.931**
R2-207-5d-100c								
Best	3401.272	3752.537	3839.877	3609.740	3652.646	3583.521	1226.683	**1072.657**
Avg.	3437.942	3769.943	3845.589	3655.904	3692.655	3604.705	1241.851	**1083.270**
R2-207-10d-100c								
Best	2658.860	2833.292	2894.221	3315.297	2873.026	2864.324	**1047.136**	1069.624
Avg.	2684.473	2846.462	2963.628	3452.572	2879.482	2892.627	1059.594	**1069.892**
R2-210-5d-100c								
Best	3161.732	3545.134	3374.468	3343.956	2875.864	3244.765	1035.636	**1014.668**
Avg.	3172.642	3579.642	3385.244	3386.862	2952.452	3352.660	1043.362	**1015.492**
R2-210-10d-100c								
Best	2325.903	3058.252	2883.262	2529.362	2355.186	2746.377	1032.622	**979.318**
Avg.	2364.644	3130.620	2915.337	2537.734	2373.733	2753.835	1043.325	**980.997**

the results. Finally, we compared each of the other methods pairwise with the HVPL. The p-values were considered as the main criteria for accepting or rejecting the null hypothesis. A larger p-values indicates weaker evidence against the null hypothesis, and conversely, a smaller p-value indicates sufficient evidence to reject the null hypothesis. As shown in Table 20, the p-values for all data sets are below 0.01, which led us to the conclusion that at significance level α equals 0.01, the null hypothesis was strongly rejected at the 99% significance level, and the optimal values of the proposed algorithm were significantly different from the optimal values of the other algorithms.

5. Conclusion

This paper presents HVPL, a hybrid algorithm developed to address the LRPSDPTW problem. In order to avoid the generation of infeasible solutions, an improved Bellman's algorithm is employed to construct the solutions. The proposed algorithm discretizes the original swarm-based volleyball premier league algorithm by redesigning a series of strategies and integrating two other algorithms with different search mechanisms. Specifically, the ACO with a global search mechanism and the ALNS with a neighborhood search mechanism are embedded

Table 17
The HVPL and other algorithms for dataset RC1.

	ALNS	DE	ACO	GA	SA	TS	LNS-ACO	HVPL
RC1-101-3d-20c								
Best	**238.343**	422.332	284.171	395.002	439.411	689.251	263.241	240.107
Avg.	**238.343**	422.332	284.171	395.002	442.520	689.251	263.241	240.107
RC1-107-5d-20c								
Best	351.062	492.673	312.283	302.064	516.521	726.464	286.100	**284.909**
Avg.	354.200	492.673	312.283	302.064	517.121	730.006	288.030	**284.909**
RC1-102-5d-50c								
Best	600.698	1736.804	912.633	1127.820	1334.813	1452.020	547.062	**441.387**
Avg.	608.923	1793.604	933.259	1145.622	1351.003	1473.122	551.007	**442.325**
RC1-108-5d-50c								
Best	1113.830	1715.633	983.222	1185.632	1212.612	1489.702	638.907	**584.828**
Avg.	1158.176	1746.311	1021.914	1197.810	1238.206	1496.226	645.163	**587.972**
RC1-103-5d-100c								
Best	1615.930	3577.301	2801.982	3002.355	2698.364	2823.622	**1126.002**	1129.882
Avg.	1643.361	3611.057	2838.173	3031.364	2706.203	2876.104	1175.320	**1134.770**
RC1-103-10d-100c								
Best	1251.037	3064.283	2503.615	2821.013	2286.241	2639.100	1031.021	**984.102**
Avg.	1269.998	3090.311	2534.465	2862.641	2320.422	2687.305	1057.381	**987.235**
RC1-106-5d-100c								
Best	1624.083	3584.631	2634.100	3435.644	2732.113	2838.923	1341.247	**1127.467**
Avg.	1647.034	3620.316	2661.634	3497.204	2766.841	2863.034	1378.809	**1136.254**
RC1-106-10d-100c								
Best	1146.316	2842.322	2443.239	2591.405	2481.247	2369.503	1061.254	**1022.621**
Avg.	1175.013	2861.347	2507.863	2620.320	2504.674	2383.016	1076.032	**1026.754**

Table 18
The HVPL and other algorithms for dataset RC2.

	ALNS	DE	ACO	GA	SA	TS	LNS-ACO	HVPL
RC2-201-3d-20c								
Best	282.632	391.774	412.026	342.434	471.881	492.109	283.471	**269.812**
Avg.	282.632	392.012	412.026	342.751	471.881	492.109	283.471	**269.812**
RC2-207-5d-20c								
Best	402.274	492.370	302.989	324.216	527.454	642.747	276.312	**253.318**
Avg.	402.274	492.370	303.620	324.216	527.454	646.012	277.235	**253.318**
RC2-202-5d-50c								
Best	463.840	1907.880	854.314	1037.301	1241.231	1418.011	498.031	**427.101**
Avg.	474.305	1975.647	880.910	1066.710	1272.820	1473.404	502.104	**427.912**
RC2-208-5d-50c								
Best	884.007	1989.211	1009.106	1620.371	1511.117	1387.045	610.414	**526.608**
Avg.	1001.112	1994.824	1062.004	1661.306	1627.997	1412.101	621.700	**528.702**
RC2-203-5d-100c								
Best	1384.053	3702.420	3069.922	3353.403	3000.974	2965.204	995.001	**927.913**
Avg.	1406.301	3720.624	3123.104	3395.077	3062.212	3102.614	1004.740	**939.013**
RC2-203-10d-100c								
Best	1064.853	3251.803	2544.027	3276.853	2558.061	3189.581	903.728	**863.998**
Avg.	1099.852	3346.832	2589.558	3317.049	2581.203	3262.671	923.626	**882.121**
RC2-206-5d-100c								
Best	1724.611	3402.410	3154.530	3413.561	3136.393	3163.086	1107.090	**1059.768**
Avg.	1821.363	3426.363	3213.551	3476.336	3163.302	3200.361	1135.420	**1064.810**
RC2-206-10d-100c								
Best	1284.566	3321.531	2642.013	3462.706	2645.189	2601.841	906.904	**897.776**
Avg.	1301.032	3372.850	2782.360	3504.421	2699.008	2739.926	952.352	**912.482**

as sub-algorithms. Additionally, two local search operators, namely the repositioning strategy and substitution strategy, are devised to enhance the solutions updated by the ACO. The framework structure seamlessly integrates the advantages of both sub-algorithms to balance exploration and exploitation. To accelerate the convergence of the algorithm and guide it towards better solution quality, a learning phase and elimination phase are designed. To evaluate the exploration and exploitation capabilities of the proposed algorithm, a comparative analysis was conducted with seven state-of-the-art algorithms. A comprehensive set of 35 benchmark functions was employed, encompassing unimodal, multimodal, and CEC-BC-2017 benchmark functions. The experimental results, supported by the Friedman mean rank test, consistently demonstrate the superior performance of HVPL compared to the other algorithms. Furthermore, when applied to the LRPSDPTW problem, HVPL achieves the optimal value in 58 out of 66 instances, surpassing widely used metaheuristics such as ALNS, DE, ACO, GA, SA, TS, as well as the hybrid metaheuristic LNS-ACO. This outcome serves as empirical evidence of the efficacy of the algorithm in solving combinatorial optimization problems.

Although the use of improved Bellman's algorithm can avoid generating a large number of ineffective solutions, the use of this method often leads to high computational costs for the HVPL algorithm. Therefore, developing a more efficient method for constructing solutions is the focus of our future work.

Table 19
The HVPL and other algorithms for dataset pr.

	ALNS	DE	ACO	GA	SA	TS	LNS-ACO	HVPL
pr-01-4d-47c								
Best	1458.475	2254.052	2133.593	1983.602	2001.747	2029.383	**1080.200**	1081.158
Avg.	1462.052	2273.721	2141.351	1992.012	2006.316	2171.869	1083.343	**1081.591**
CPU(s)	687.540	82.730	24.020	128.360	22.180	47.300	1434.560	1725.310
pr-02-4d-95c								
Best	3029.216	5219.421	5691.650	4743.841	4427.612	4462.008	1992.918	**1863.296**
Avg.	3096.275	5273.462	5720.028	4800.411	4460.831	4531.531	2005.383	**1863.296**
CPU(s)	1382.670	191.280	67.430	330.450	62.430	89.300	2034.560	2780.640
pr-03-4d-143c								
Best	3914.367	9521.930	8993.427	8502.515	6683.428	7427.423	2637.360	**2599.282**
Avg.	4012.940	9601.366	9136.009	8643.709	6723.087	7502.805	2703.400	**2616.103**
CPU(s)	4723.560	280.460	138.500	508.640	130.260	153.200	14 310.670	15 304.040
pr-04-4d-191c								
Best	4503.673	11 239.795	11 585.008	10 079.935	8378.581	9159.934	3272.305	**3034.262**
Avg.	4703.519	12 351.115	11 692.773	10 335.536	8482.603	9361.686	3347.429	**3057.385**
CPU(s)	5739.240	347.640	240.000	664.320	244.740	355.080	9894.400	11 920.510
pr-05-5d-238c								
Best	5121.044	14 527.324	12 992.466	12 267.427	9211.657	10 466.727	3411.463	**3384.217**
Avg.	5356.157	14 842.594	13 361.006	12 788.805	9884.001	10 994.030	3612.069	**3384.682**
CPU(s)	39 083.450	527.680	373.020	752.710	458.090	787.350	48 366.200	57 976.300
pr-06-5d-287c								
Best	6725.788	16 750.084	17 214.705	16 838.200	124 221.551	13 092.560	4079.940	**4043.645**
Avg.	7109.709	17 197.971	17 349.639	17 661.969	126 917.647	13 541.094	4278.973	**4066.056**
CPU(s)	45 800.640	553.670	511.320	996.370	544.200	1566.409	64 163.570	78 352.800

Table 20
p-values of the Wilcoxon Signed-Rank Test ($\alpha = 0.01$).

	Instances						
	Set C1	Set C2	Set R1	Set R2	Set RC1	Set RC2	Set pr
HVPL vs. ALNS	3.39E−18	9.10E−19	2.82E−18	9.87E−19	8.08E−13	3.07E−18	3.07E−18
HVPL vs. ACO	3.53E−18	3.71E−18	3.31E−18	2.74E−18	1.63E−18	2.10E−18	8.18E−20
HVPL vs. DE	3.31E−18	3.17E−18	3.22E−18	2.94E−18	5.48E−20	1.98E−18	4.66E−20
HVPL vs. GA	3.65E−18	2.62E−18	3.12E−18	3.65E−18	3.39E−18	3.76E−18	3.60E−18
HVPL vs. SA	3.74E−18	3.52E−18	1.41E−18	3.76E−18	2.85E−18	3.35E−18	2.66E−18
HVPL vs. TS	3.59E−18	3.39E−18	3.69E−18	3.64E−18	1.08E−19	2.58E−18	5.93E−19
HVPL vs. LNS-ACO	3.23E−18	9.08E−19	1.33E−18	9.87E−19	3.27E−21	1.33E−18	1.16E−18

Table 21
The parameter values of the comparative algorithms.

Algorithm	Parameter value
ALNS	Cooling rate = 0.9; Frequency of weight adjustment = 5; Start temperature = 4500
ACO	Population size = 10; Speed of evaporation = 0.1; Initial pheromone = 10
DE	Population size = 10; Scaling factor = 0.5; Crossover probability = 0.6
GA	Population size = 10; Crossover probability = 0.9; Mutation probability = 0.2
SA	Initial temperature = 6000; Final temperature = 0.001; Maximum allowable number = 100; Boltzmann constant = 0.1
TS	Max-swap-loc-d = 4; Max-add-loc-d = 4; Max-insert-route = 6; Max-swap-route = 8
LNS-ACO	Relative importance of pheromone intensity versus information = (1, 1); ψ = 1.75

CRediT authorship contribution statement

Shuo Sun: Analyzed the problem, Designed the method, Analyzed and interpreted data, Performed the experiments, Writing – original draft, Investigation, Validation. **Liang Ma:** Conceptualization, Methodology, Writing – review & editing, Supervision. **Yong Liu:** Methodology, Supervision, Writing – review & editing. **Chunjian Shang:** Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This research is supported by the project of the humanities and social sciences of the Ministry of Education of China under Grant (No. 16YJA630037), the key project of soft science research in science and technology innovation action plan of Shanghai, China under Grant(No. 18692110500), and the philosophy and social science planning project of Shanghai, China under Grant (No. 2019BGL014).

Appendix

See Table 21.

References

[1] Caroline Prodhon, Christian Prins, A survey of recent research on location-routing problems, European J. Oper. Res. 238 (1) (2014) 1–17.
[2] F.E. Maranzana, On the location of supply points to minimize transport costs, J. Oper. Res. Soc. 15 (3) (1964) 261–270.

[3] Said Salhi, Graham K. Rand, The effect of ignoring routes when locating depots, European J. Oper. Res. 39 (2) (1989) 150–156.
[4] Özge Şatir Akpunar, Şener Akpinar, A hybrid adaptive large neighbourhood search algorithm for the capacitated location routing problem, Expert Syst. Appl. 168 (2021) 114304.
[5] Michael Drexl, Michael Schneider, A survey of variants and extensions of the location-routing problem, European J. Oper. Res. 241 (2) (2015) 283–308.
[6] Kannan Govindan, Hamed Soleimani, Devika Kannan, Reverse logistics and closed-loop supply chain: A comprehensive review to explore the future, European J. Oper. Res. 240 (3) (2015) 603–626.
[7] Mohammed Alkahtani, Aiman Ziout, Bashir Salah, Moath Alatefi, Abd Elatty E Abd Elgawad, Ahmed Badwelan, Umar Syarif, An insight into reverse logistics with a focus on collection systems, Sustainability 13 (2) (2021) 548.
[8] Hsiao-Fan Wang, Ying-Yen Chen, A genetic algorithm for the simultaneous delivery and pickup problems with time window, Comput. Ind. Eng. 62 (1) (2012) 84–95.
[9] Sener Akpinar, Hybrid large neighbourhood search algorithm for capacitated vehicle routing problem, Expert Syst. Appl. 61 (2016) 28–38.
[10] Ran Liu, Zhibin Jiang, A hybrid large-neighborhood search algorithm for the cumulative capacitated vehicle routing problem with time-window constraints, Appl. Soft Comput. 80 (2019) 18–30.
[11] T.O. Ting, Xin-She Yang, Shi Cheng, Kaizhu Huang, Hybrid metaheuristic algorithms: past, present, and future, in: Recent Advances in Swarm Intelligence and Evolutionary Computation. Vol. 585, Springer, 2015, pp. 71–83.
[12] Xuefeng Wang, Fang Yang, Dawei Lu, Multi-objective location-routing problem with simultaneous pickup and delivery for urban distribution, J. Intell. Fuzzy Systems 35 (4) (2018) 3987–4000.
[13] Feiyue Qiu, Guodao Zhang, Ping-Kuo Chen, Cheng Wang, Yi Pan, Xin Sheng, Dewei Kong, A novel multi-objective model for the cold chain logistics considering multiple effects, Sustainability 12 (19) (2020) 8068.
[14] Longlong Leng, Yanwei Zhao, Zheng Wang, Hongwei Wang, Jingling Zhang, Shared mechanism-based self-adaptive hyperheuristic for regional low-carbon location-routing problem with time windows, Math. Probl. Eng. 2018 (2018).
[15] F. Yu Vincent, Shih-Wei Lin, Wenyih Lee, Ching-Jung Ting, A simulated annealing heuristic for the capacitated location routing problem, Comput. Ind. Eng. 58 (2) (2010) 288–299.
[16] Lin Zhou, Xu Wang, Lin Ni, Yun Lin, Location-routing problem with simultaneous home delivery and customer's pickup for city distribution of online shopping purchases, Sustainability 8 (8) (2016) 828.
[17] Elham Jelodari Mamaghani, Soheil Davari, The bi-objective periodic closed loop network design problem, Expert Syst. Appl. 144 (2020) 113068.
[18] Naser Moosavian, Babak Kasaee Roodsari, Soccer league competition algorithm: A novel meta-heuristic algorithm for optimal design of water distribution networks, Swarm Evol. Comput. 17 (2014) 14–24.
[19] Tanawat Worawattawechai, Boonyarit Intiyot, Chawalit Jeenanunta, William G Ferrell Jr., A learning enhanced golden ball algorithm for the vehicle routing problem with backhauls and time windows, Comput. Ind. Eng. 168 (2022) 108044.
[20] HREH Bouchekara, Most Valuable Player Algorithm: a novel optimization algorithm inspired from sport, Oper. Res. 20 (1) (2020) 139–195.
[21] Ali Husseinzadeh Kashan, League Championship Algorithm (LCA): An algorithm for global optimization inspired by sport championships, Appl. Soft Comput. 16 (2014) 171–200.
[22] Reza Moghdani, Khodakaram Salimifard, Volleyball premier league algorithm, Appl. Soft Comput. 64 (2018) 161–185.
[23] Reza Moghdani, Mohamed Abd Elaziz, Davood Mohammadi, Nabil Neggaz, An improved volleyball premier league algorithm based on sine cosine algorithm for global optimization problem, Eng. Comput. 37 (4) (2021) 2633–2662.
[24] Abhineet Prakash, Sariki Murali, Ravi Shankar, Ravi Bhushan, HVDC tie-link modeling for restructured AGC using a novel fractional order cascade controller, Electr. Power Syst. Res. 170 (2019) 244–258.
[25] Reza Moghdani, Khodakaram Salimifard, Emrah Demir, Abdelkader Benyettou, Multi-objective volleyball premier league algorithm, Knowl.-Based Syst. 196 (2020) 105781.
[26] Khodakaram Salimifard, Jingpeng Li, Davood Mohammadi, Reza Moghdani, A multi objective volleyball premier league algorithm for green scheduling identical parallel machines with splitting jobs, Appl. Intell. 51 (2021) 4143–4161.
[27] Mohamed Abd Elaziz, Neggaz Nabil, Reza Moghdani, Ahmed A Ewees, Erik Cuevas, Songfeng Lu, Multilevel thresholding image segmentation based on improved volleyball premier league algorithm using whale optimization algorithm, Multimedia Tools Appl. 80 (8) (2021) 12435–12468.
[28] Sariki Murali, Ravi Shankar, Exploration of novel optimal fuzzy-based controller for enhancement of frequency regulation of deregulated hybrid power system with modified HVDC tie-line, Int. J. Intell. Syst. 37 (7) (2022) 4163–4189.
[29] Murali Sariki, Ravi Shankar, Optimal CC-2DOF (PI)-PDF controller for LFC of restructured multi-area power system with IES-based modified HVDC tie-line and electric vehicles, Eng. Sci. Technol. Int. J. 32 (2022) 101058.
[30] Francisco J Rodriguez, Carlos Garcia-Martinez, Manuel Lozano, Hybrid metaheuristics based on evolutionary algorithms and simulated annealing: taxonomy, comparison, and synergy test, IEEE Trans. Evol. Comput. 16 (6) (2012) 787–800.
[31] Kun He, Kevin Tole, Fei Ni, Yong Yuan, Linyun Liao, Adaptive large neighborhood search for solving the circle bin packing problem, Comput. Oper. Res. 127 (2021) 105140.
[32] Komgrit Leksakul, Uttapol Smutkupt, Raweeroj Jintawiwat, Suriya Phongmoo, Heuristic approach for solving employee bus routes in a large-scale industrial factory, Adv. Eng. Inform. 32 (2017) 176–187.
[33] Hongbo Li, Xiaoxia Zhang, Shuai Fu, Yinyin Hu, A hybrid algorithm based on ant colony optimization and differential evolution for vehicle routing problem, Eng. Lett. 29 (3) (2021).
[34] Christian Prins, A simple and effective evolutionary algorithm for the vehicle routing problem, Comput. Oper. Res. 31 (12) (2004) 1985–2002.
[35] John E. Beasley, Route first—cluster second methods for vehicle routing, Omega 11 (4) (1983) 403–408.
[36] Marco Dorigo, Vittorio Maniezzo, Alberto Colorni, Ant system: Optimization by a colony of cooperating agents, IEEE Trans. Syst. Man Cybern. B 26 (1) (1996) 29–41.
[37] Shan-Huen Huang, Ying-Hua Huang, Carola A Blazquez, Germán Paredes-Belmar, Application of the ant colony optimization in the resolution of the bridge inspection routing problem, Appl. Soft Comput. 65 (2018) 443–461.
[38] Fanlei Yan, Autonomous vehicle routing problem solution based on artificial potential field with parallel ant colony optimization (ACO) algorithm, Pattern Recognit. Lett. 116 (2018) 195–199.
[39] Rico Wijaya Dewantoro, Poltak Sihombing, et al., The combination of ant colony optimization (ACO) and tabu search (TS) algorithm to solve the traveling salesman problem (TSP), in: 2019 3rd International Conference on Electrical, Telecommunication and Computer Engineering, ELTICOM, IEEE, 2019.
[40] Jun Wu, Mingjie Fan, Yang Liu, Yupeng Zhou, Nan Yang, Minghao Yin, A hybrid ant colony algorithm for the winner determination problem, Math. Biosci. Eng. 19 (3) (2022) 3202–3222.
[41] HAYDER NASER KHRAIBET Al-behadili, KR Ku-Mahamud, RAFID Sagban, Hybrid ant colony optimization and iterated local search for rules-based classification, J. Theor. Appl. Inf. Technol. 98 (04) (2020) 657–671.
[42] Surendra Reddy Kancharla, Gitakrishnan Ramadurai, Electric vehicle routing problem with non-linear charging and load-dependent discharging, Expert Syst. Appl. 160 (2020) 113714.
[43] Stefan Ropke, David Pisinger, An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows, Transp. Sci. 40 (4) (2006) 455–472.
[44] Scott Kirkpatrick, C. Daniel Gelatt Jr., Mario P. Vecchi, Optimization by simulated annealing, science 220 (4598) (1983) 671–680.
[45] Stefan Ropke, David Pisinger, An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows, Transp. Sci. 40 (4) (2006) 455–472.
[46] Benjamin Doerr, Daniel Johannsen, Timo Kötzing, Frank Neumann, Madeleine Theile, More effective crossover operators for the all-pairs shortest path problem, Theoret. Comput. Sci. 471 (2013) 12–26.
[47] Guangming Lin, Xin Yao, Analysing crossover operators by search step size, in: Proceedings of 1997 IEEE International Conference on Evolutionary Computation, ICEC'97, IEEE, 1997.
[48] Marius M. Solomon, Algorithms for the vehicle routing and scheduling problems with time window constraints, Oper. Res. 35 (2) (1987) 254–265.
[49] Stefan Ropke, David Pisinger, An adaptive large neighborhood search heuristic for the pickup and delivery problem with time windows, Transp. Sci. 40 (4) (2006) 455–472.
[50] Shan-Huen Huang, Ying-Hua Huang, Carola A Blazquez, Germán Paredes-Belmar, Application of the ant colony optimization in the resolution of the bridge inspection routing problem, Appl. Soft Comput. 65 (2018) 443–461.
[51] William C. Parr, Introduction to Quality Engineering: Designing Quality into Products and Processes, Taylor & Francis, 1989.
[52] Ashkan Mozdgir, Iraj Mahdavi, Iman Seyedi Badeleh, Maghsud Solimanpur, Using the taguchi method to optimize the differential evolution algorithm parameters for minimizing the workload smoothness index in simple assembly line balancing, Math. Comput. Modelling 57 (1–2) (2013) 137–151.
[53] Shi-Yi Tan, Wei-Chang Yeh, The vehicle routing problem: State-of-the-art classification and review, Appl. Sci. 11 (21) (2021) 10295.
[54] Mohd Herwan Sulaiman, Zuriani Mustaffa, Mohd Mawardi Saari, Hamdan Daniyal, Barnacles Mating Optimizer: A new bio-inspired algorithm for solving engineering optimization problems, Eng. Appl. Artif. Intell. 87 (2020) 103330.
[55] Qamar Askari, Irfan Younas, Mehreen Saeed, Political Optimizer: A novel socio-inspired meta-heuristic for global optimization, Knowl.-Based Syst. 195 (2020) 105709.
[56] Iman Ahmadianfar, Omid Bozorg-Haddad, Xuefeng Chu, Gradient-based optimizer: A new metaheuristic optimization algorithm, Inform. Sci. 540 (2020) 131–159.
[57] Afshin Faramarzi, Mohammad Heidarinejad, Brent Stephens, Seyedali Mirjalili, Equilibrium optimizer: A novel optimization algorithm, Knowl.-Based Syst. 191 (2020) 105190.
[58] Weiguo Zhao, Zhenxing Zhang, Liying Wang, Manta ray foraging optimization: An effective bio-inspired optimizer for engineering applications, Eng. Appl. Artif. Intell. 87 (2020) 103300.

[59] Jui-Sheng Chou, Ngoc-Mai Nguyen, FBI inspired meta-optimization, Appl. Soft Comput. 93 (2020) 106339.
[60] Abdulaziz Alorf, A survey of recently developed metaheuristics and their comparative analysis, Eng. Appl. Artif. Intell. 117 (2023) 105622.
[61] Milton Friedman, A comparison of alternative tests of significance for the problem of m rankings, Ann. Math. Stat. 11 (1) (1940) 86–92.
[62] Pavel Krömer, Ajith Abraham, Václav Snášel, Eshetie Berhan, Daniel Kitaw, On the differential evolution for vehicle routing problem, in: 2013 International Conference on Soft Computing and Pattern Recognition, SoCPaR, IEEE, 2013.
[63] David Pisinger, Stefan Ropke, A general heuristic for vehicle routing problems, Comput. Oper. Res. 34 (8) (2007) 2403–2435.
[64] Sener Akpinar, Hybrid large neighbourhood search algorithm for capacitated vehicle routing problem, Expert Syst. Appl. 61 (2016) 28–38.
[65] Pavel Krömer, Ajith Abraham, Václav Snášel, Eshetie Berhan, Daniel Kitaw, On the differential evolution for vehicle routing problem, in: 2013 International Conference on Soft Computing and Pattern Recognition, SoCPaR, IEEE, 2013.
[66] John E. Bell, Patrick R. McMullen, Ant colony optimization techniques for the vehicle routing problem, Adv. Eng. Inf. 18 (1) (2004) 41–48.